

SVEUČILIŠTE U ZAGREBU
FAKULTET ELEKTROTEHNIKE I RAČUNARSTVA

ZAVRŠNI RAD br. 2059

**SUPARNIČKI NAPADI NA MODELE ZA
KLASIFIKACIJU SLIKA**

Tonka Šegvić

Zagreb, lipanj, 2025.

Zagreb, 3. ožujka 2025.

ZAVRŠNI ZADATAK br. 2059

Pristupnica: **Tonka Šegvić (0036546553)**
Studij: Elektrotehnika i informacijska tehnologija i Računarstvo
Modul: Računarstvo
Mentor: prof. dr. sc. Domagoj Jakobović

Zadatak: **Suparnički napadi na modele za klasifikaciju slika**

Opis zadatka:

Opisati i istražiti vrste neprijateljskih napada na klasifikacijske modele. Posebnu pažnju posvetiti postojećim napadima na modele za klasifikaciju slike i osvrnuti se na njihovu robusnost. Proučiti napad temeljen na gradijentima neuronske mreže (engl. Fast Gradient Signed Method) i njegovu primjenu u stvaranju napada. Ostvariti programski sustav za stvaranje i primjenu napada na modele za klasifikaciju slike koristeći odabrane algoritme. Ispitati rad implementacije s obzirom na kriterije optimizacije te usporediti rezultate dobivene primjenom odabranih algoritama. Radu priložiti izvorne tekstove programa, dobivene rezultate uz potrebna objašnjenja i korištenu literaturu.

Rok za predaju rada: 23. lipnja 2025.

Zahvaljujem se mentoru Domaguju Jakoboviću na korisnim savjetima i podršci tijekom izrade ovog završnog rada. Također, posebnu zahvalu upućujem svojoj obitelji i prijateljima na razumijevanju, motivaciji i podršci tijekom cijelog studija.

Sadržaj

1. Uvod	4
2. Općenito o dubokom učenju	5
2.1. Potpuno povezani modeli	5
2.2. Konvolucijski modeli	7
2.2.1. Resnet-20	8
2.3. Funkcije gubitka	10
2.3.1. Srednja kvadratna greška	10
2.3.2. Unakrsna entropija	10
2.3.3. Negativna logaritamska izglednost	11
2.4. Optimizacijski postupci	11
2.4.1. Genetski algoritmi	12
2.4.2. Newtonov algoritam	12
2.4.3. Gradijentni spust	12
2.4.4. Stohastički gradijentni spust	13
3. Suparnički napadi	15
3.1. Tipovi suparničkih napada	16
3.1.1. White-box i Black-box napadi	16
3.1.2. Ciljani i neciljani napadi	16
3.2. Napad FGSM	17
3.2.1. Varijante FGSM-a.	19
3.3. Ostale popularne metode napada	19
3.3.1. Napad projiciranim gradijentnim spustom (Projected Gradient Descent - PGD)	19

3.3.2.	Carlini & Wagner napad	20
3.3.3.	One Pixel Attack	21
3.4.	Obrane od suparničkih napada	22
4.	Implementacija i rezultati	23
4.1.	Razvojno okruženje	23
4.1.1.	Python	23
4.1.2.	PyTorch	23
4.1.3.	Skup podataka MNIST	24
4.1.4.	Skup podataka CIFAR-100	24
4.2.	Treniranje modela na skupu podataka MNIST	26
4.2.1.	Arhitektura mreže	26
4.2.2.	Optimizacija parametara	27
4.3.	Model za klasifikaciju CIFAR-100 skupa podataka	30
4.4.	FGSM implementacija	31
4.4.1.	Odvajanje logike modela i napada	31
4.4.2.	Razlike između MNIST i CIFAR-100 strategije	31
4.4.3.	Logika napada	32
4.4.4.	Ciljani i neciljani napad	34
4.5.	Rezultati za potpuno povezani model i MNIST	35
4.5.1.	Neciljani napad	35
4.5.2.	Ciljani napad	36
4.6.	Rezultati za resnet20 model i CIFAR-100	38
4.6.1.	Neciljani napad	38
4.6.2.	Ciljani napad	39
4.7.	Usporedba robusnosti	42
4.7.1.	Neciljani napad	42
4.7.2.	Ciljani napad	44
4.8.	Nedovršeni pokušaji i napuštene ideje	45
5.	zaključak	47
	Literatura	48

Sažetak	51
Abstract	52

1. Uvod

Duboke neuronske mreže danas čine temelj brojnih sustava umjetne inteligencije, osobito u području računalnog vida. Međutim, iako ovi modeli postižu dobre rezultate u klasičkim zadacima, pokazalo se da su osjetljivi na pažljivo konstruirane male perturbacije koje mogu drastično promijeniti njihov izlaz. Takve ulaze nazivamo suparničkim primjerima, a metode za njihovo generiranje predstavljaju suparničke napade.

Suparnički napadi predstavljaju ozbiljan sigurnosni izazov, posebno u primjenama gdje se od modela očekuje robusnost – primjerice u autonomnim vozilima, medicinskoj dijagnostici ili sustavima prepoznavanja lica. Cilj ovog rada je istražiti različite metode suparničkih napada s naglaskom na Fast Gradient Sign Method (FGSM).

U praktičnom dijelu rada izrađena je i isprobana programska implementacija koja omogućuje izvođenje FGSM napada nad različitim modelima i skupovima podataka. Osim implementacije FGSM-a, implementiran je i algoritam stohastičkog gradijentnog spusta (SGD), korišten za treniranje potpuno povezanog modela na skupu MNIST.

Rad je organiziran tako da se prvo pokrije teorijska osnova dubokih modela i suparničkih napada te se u kasnijim poglavljima usredotočuje na provedene eksperimente te analizira njihove rezultate.

2. Općenito o dubokom učenju

Duboko učenje ubrzalo je napredak u mnogim područjima umjetne inteligencije, od prepoznavanja slika do prirodnog jezika.

Temelji dubokog učenja postavljeni su 1980-ih kada su Rumelhart i suradnici uveli algoritam propagiranja unatrag (backpropagation) [1] koji je omogućio učinkovito računanje gradijenata u višeslojnim mrežama. LeCun je potom razvio konvolucijske mreže [2], a 2012. godine je Krizhevsky na natjecanju ImageNet pokazao da duboke mreže mogu značajno nadmašiti postojeće algoritme [3].

Svaki algoritam strojnog učenja sastoji se od modela, funkcije gubitka i optimizacijskog postupka. Model predstavlja željenu funkciju sa slobodnim parametrima koji se naštimaavaju tijekom učenja. Funkcija gubitka je mjera kvalitete ponašanja modela. Tijekom optimizacijskog postupka parametri modela se iterativno ažuriraju s ciljem minimizacije funkcije gubitka.

U ovom radu razmatramo nadzirano učenje kod kojeg za svaki podatak x postoji oznaka y koja odgovara željenom izlazu modela.

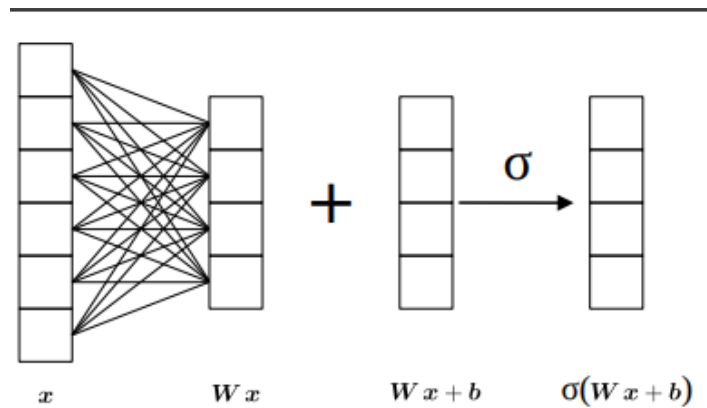
2.1. Potpuno povezani modeli

Potpuno povezani modeli sastoje se od slijeda potpuno povezanih slojeva. U ovim modelima svaki neuron u jednom sloju povezan je sa svakim neuronom u sljedećem sloju. Izlaz potpuno povezanog sloja može se opisati kao:

$$y = \sigma(Wx + b) \quad (2.1)$$

U prikazanoj jednadžbi, W je matrica težina, x ulazni vektor, b vektor pomaka (bias), a σ nelinearna aktivacijska funkcija.

Generiranje izlaza u potpuno povezanom sloju (2.1) možemo slikovito prikazati na sljedeći način:



Slika 2.1. Potpuno povezani sloj. Preuzeto iz [4]

Potpuno povezani duboki model sastoji se od ulaznog sloja koji odgovara ulaznom vektoru, proizvoljnog broja skrivenih slojeva te izlaznog sloja koji odgovara rezultatu funkcije koju aproksimiramo modelom.

Dakle, arhitektura potpuno povezanog dubokog modela određena je brojem potpuno povezanih slojeva, dimenzionalnošću izlaza svakog sloja te aktivacijskom funkcijom.

Za aktivacijsku funkciju se uzima neka nelinearna funkcija. Često korišteni primjeri su: ReLU (zglobnica), sigmoida, tangens hiperbolni i drugi.

ReLU funkcija najčešći je odabir za aktiviranje skrivenih slojeva dubokih modela, a definira se sljedećom jednadžbom:

$$\text{ReLU}(x) = \max(0, x) \quad (2.2)$$

Skup svih težina i pomaka u svim slojevima nazivamo parametrima modela θ .

Potpuno povezani modeli često se koriste za zadatke klasifikacije i regresije, ali zbog velikog broja parametara mogu biti skloni prenaučivosti (overfitting) na složenijim zadacima.

Neki eksperimenti u ovom radu koriste potpuno povezani model upravo zbog njegove jednostavnosti, ali i činjenice da se pokazao dovoljno dobrim za relativno jednostavne zadatke.

2.2. Konvolucijski modeli

Za razliku od potpuno povezanog modela, konvolucijski model iskorištava prostornu strukturu ulaznih podataka tako da neuroni u jednom sloju nisu povezani sa svim neuronima u prethodnom sloju, već samo s lokalnim regijama. Ova lokalnost omogućuje manji broj parametara, bolju skalabilnost i robusnost na pomake u ulazu. Svaki kanal koristi dijeljene parametre (K i b). Umjesto množenja cijelog ulaznog vektora matricom težina, kao u potpuno povezanim slojevima, konvolucijski sloj koristi operaciju konvolucije :

$$Y = \sigma(X * K + b) \quad (2.3)$$

Konvolucija (*) je matematička operacija kojom se mala matrica težina koju nazivamo konvolucijska jezgra K , pomiče po ulaznom podatku X (npr. slici), pri čemu se na svakoj poziciji računa suma umnožaka između elemenata jezgre i odgovarajućih elemenata ulaza. Rezultat je nova matrica, koja sadrži lokalno sažete informacije o ulazu.

$$(X * K)(i, j) = \sum_m \sum_n X(i + m, j + n) \cdot K(m, n) \quad (2.4)$$

Gornja jednadžba predstavlja operaciju konvolucije za jednokanalni ulaz¹. Za višekanalni ulaz (npr. RGB sliku) konvolucija se računa zasebno po svakom kanalu, a zatim se rezultati sumiraju, što je konceptualno slično, ali se jednadžba tada proširuje na više dimenzija.

Svaka jezgra uči prepoznati određeni uzorak poput rubova, tekstura ili oblika u ulaznim podacima. Budući da se ista jezgra primjenjuje na svim pozicijama ulaza u jednom izlaznom kanalu, težine se dijele, čime se znatno smanjuje broj parametara. Na taj način konvolucijski modeli učinkovito uče značajke koje su invarijantne na translaciju i ponav-

¹Strogo matematički gledano, jednadžba 2.4 prikazuje unakrsnu korelaciju koja se u praksi koristi češće zbog bolje učinkovitosti. Matematička konvolucija umjesto $i + m$ ima $i - m$.

ljaju se u prostoru: konvolucija transliranog ulaza odgovara transliranoj konvoluciji originalnog ulaza.

Nakon konvolucijskih slojeva često slijedi sloj sažimanja. Sloj sažimanja smanjuje dimenzije ulaznih podataka sažimanjem lokalnih regija. Najčešće se koristi sažimanje maksimumom (max pooling) koje na lokaciji (i, j) uzima maksimum unutar regije R_{ij} :

$$Y(i, j) = \max_{(m,n) \in R_{ij}} X(m, n) \quad (2.5)$$

Nakon nekoliko slojeva konvolucije i sažimanja često slijedi potpuno povezani sloj koji donosi konačnu klasifikacijsku ili regresijsku odluku.

2.2.1. Resnet-20

ResNet (Residual Network) je arhitektura konvolucijskih mreža koju su 2015. godine predstavili He i suradnici [5]. Glavna inovacija ResNet-a je uvođenje rezidualnih veza koje omogućavaju treniranje vrlo dubokih mreža bez problema vezanih uz slabu konvergenciju na skupu za učenje. Rezidualni modeli sastoje se od rezidualnih jedinica F koje možemo opisati sljedećom jednačinom [6]:

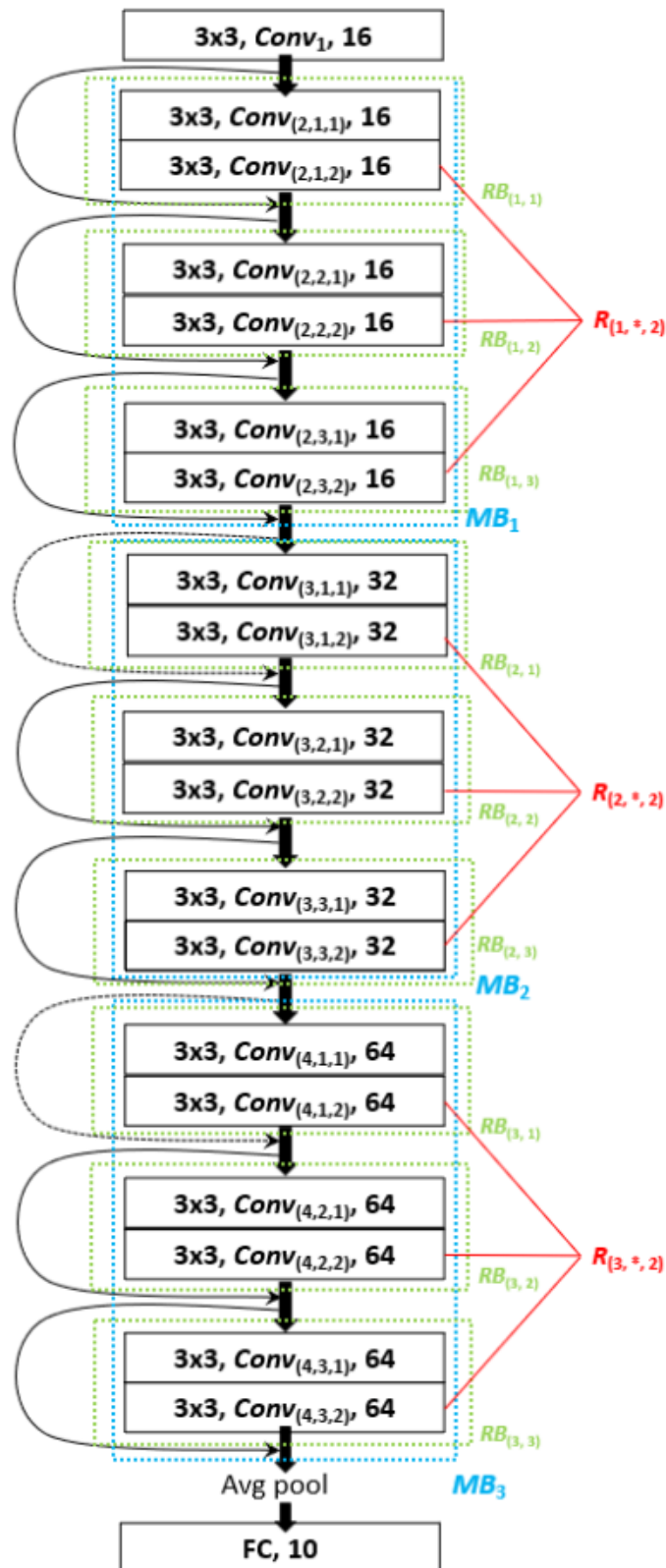
$$y = F(x) + x \quad (2.6)$$

U prikazanoj jednačini, x predstavlja ulaz rezidualne jedinice, F funkciju sa slobodnim parametrima, a y izlaz jedinice.

Prije ResNet-a, povećavanje dubine mreže često je rezultiralo pogoršanjem performansi zbog slabog napredovanja optimizacije na skupu za učenje - vrlo duboke obične mreže tipično ostvaruju veću grešku na skupu za učenje od plićih mreža.

ResNet20 je mala varijanta ResNet arhitekture koja se često koristi za učenje klasifikacijskih modela na skupovima CIFAR-10 i CIFAR-100. Sastoji se od početnog konvolucijskog sloja, tri grupe rezidualnih blokova (s 16, 32 i 64 kanala), globalnog sažimanja i potpuno povezanog sloja za klasifikaciju. Svaka grupa se sastoji od 3 rezidualna bloka. Svaki rezidualni blok sadrži dva konvolucijska sloja 3×3 s normalizacijom po grupi i

ReLU aktivacijama, te rezidualnu vezu koja izlaz zbraja s ulazom.



Slika 2.2. Arhitektura modela resnet20. Preuzeto iz [7]

ResNet20 predstavlja dobru ravnotežu između složenosti modela i računskih zahtjeva, omogućavajući relativno lako optimiziranje i dobre performanse na zadacima kla-

sifikacije slika.

2.3. Funkcije gubitka

Kao što je ranije navedeno funkcija gubitka je mjera različitosti željenog i ostvarenog ponašanja modela. To znači da gubitak mjeri razliku između dobivenih izlaza modela te oznaka (željenih izlaza) za svaki ulaz iz skupa za učenje. Različite funkcije gubitka različito penaliziraju određene tipove grešaka i zbog toga mogu biti više ili manje pogodne za specifičan oblik problema.

Navest ćemo neke često korištene funkcije gubitka.

2.3.1. Srednja kvadratna greška

Srednja kvadratna greška (Mean Squared Error - MSE) je jedna od najčešće korištenih funkcija gubitka za probleme regresije:

$$MSE = \frac{1}{n} \sum_{i=1}^n ||y_i - \hat{y}_i||^2 \quad (2.7)$$

n je broj uzoraka, y_i stvarni vektor izlaza za i -ti uzorak, $\hat{y}_i$ predviđeni vektor izlaza za i -ti uzorak, a $|| \cdot ||^2$ kvadrat euklidske norme.

2.3.2. Unakrsna entropija

Unakrsna entropija (Cross-entropy) je standardna funkcija gubitka za probleme klasifikacije:

$$H = -\frac{1}{n} \sum_{i=1}^n \sum_{c=1}^C y_{i,c} \log(\hat{y}_{i,c}) \quad (2.8)$$

$y_{i,c}$ je stvarna vjerojatnost da i -ti uzorak pripada klasi c (vektor oznake za i -ti ulaz na indexu c), $\hat{y}_{i,c}$ predviđena vjerojatnost da i -ti uzorak pripada klasi c (vektor izlaza za i -ti ulaz na indexu c), a C broj klasa.

Unakrsna entropija radi s tvrdim oznakama (hard labels) gdje je $y_{i,c} \in \{0, 1\}$ i točno

jedna klasa ima vrijednost 1, kao i s mekim oznakama (soft labels) gdje $y_{i,c} \in [0, 1]$ predstavlja vjerojatnosnu distribuciju preko klasa.

2.3.3. Negativna logaritamska izglednost

Negativna logaritamska izglednost (Negative Log-Likelihood – NLL) je često korištena funkcija gubitka za klasifikacijske modele i definira se kao:

$$NLL = -\frac{1}{n} \sum_{i=1}^n \log p_i \quad (2.9)$$

gdje je n broj uzoraka, a p_i vjerojatnost koju model dodjeljuje ispravnoj oznaci za i -ti uzorak.

Neka je $y_i = [y_{i,1}, \dots, y_{i,C}]$ stvarna oznaka za i -ti uzorak (one-hot ili distribucija), $\hat{y}_i = [\hat{y}_{i,1}, \dots, \hat{y}_{i,C}]$ predviđena distribucija vjerojatnosti po klasama, te C broj klasa.

Tada se p_i odgovara očekivanju vjerojatnosti koju model dodjeljuje točnom razredu::

$$p_i = \sum_{c=1}^C y_{i,c} \cdot \hat{y}_{i,c} \quad (2.10)$$

Kao i unakrsna entropija(2.3.2.) negativna logaritamska izglednost podržava korištenje tvrdih i mekih oznaka. Međutim, u praksi se s mekim oznakama koristi unakrsna entropija jer ima bolja teorijska svojstva.

2.4. Optimizacijski postupci

Tijekom procesa treniranja dubokih modela, cilj je pronaći optimalne vrijednosti parametara modela θ koje minimiziraju funkciju gubitka $\mathcal{L}(\theta)$. Za taj cilj koristi se niz različitih optimizacijskih algoritama. U ovom poglavlju opisani su neki od poznatijih pristupa koji se koriste za ažuriranje parametara tijekom učenja.

2.4.1. Genetski algoritmi

Genetski algoritmi temelje se na principima biološke evolucije. Svako moguće rješenje problema (tj. određena instanca parametara θ) smatra se jedinkom u populaciji. Glavna prednost ovakvih algoritama je to da ne zahtijevaju računanje gradijenata, što omogućava optimizaciju funkcija koje nisu diferencijabilne.

Postoje različite varijante genetskih algoritama, no općenito se algoritam može opisati kao niz sljedećih koraka:

1. Inicijalizacija populacije slučajnim jedinkama.
2. Evaluacija svake jedinke prema funkciji dobrote (fitness) - često negativna ili recipročna vrijednost funkcije gubitka $-\mathcal{L}(\theta)$.
3. Selekcija najboljih jedinki.
4. Križanje (crossover) i mutacija kako bi se stvorila nova generacija.

Postupak se ponavlja sve dok se ne postigne zadovoljavajuće rješenje.

2.4.2. Newtonov algoritam

Newtonov algoritam koristi drugu derivaciju (Hesseovu matricu) kako bi ubrzao konvergenciju prema minimumu funkcije gubitka. U praksi, zbog visoke složenosti izračuna Hesseove matrice kod modela s velikim brojem parametara, Newtonov algoritam se rijetko koristi u dubokom učenju. Međutim, njegovi principi su osnova za mnoge napredne varijante optimizacijskih algoritama.

2.4.3. Gradijentni spust

Gradijentni spust (ili batch gradijentni spust) jedan je od osnovnih algoritama optimizacije. U svakoj iteraciji računa se gradijent funkcije gubitka s obzirom na parametre modela, te se parametri ažuriraju u smjeru suprotnom od gradijenta. Taj smjer minimizira funkciju gubitka pod pretpostavkom da je pomak η dovoljno malen te da se viši članovi Taylorove aproksimacije mogli zanemariti.

Za ažuriranje parametara koristi se pravilo:

$$\theta' = \theta - \eta \nabla_{\theta} \mathcal{L}(\theta) \quad (2.11)$$

gdje je θ vektor parametara modela, η stopa učenja (engl. learning rate), a $\nabla_{\theta} \mathcal{L}(\theta)$ gradijent funkcije gubitka s obzirom na parametre.

Kod gradijentnog spusta, računa se gradijent funkcije gubitka za cijeli skup podataka, i zatim se parametri modela ažuriraju prema navedenom pravilu 2.11. Svako računanje gradijenta i ažuriranje parametara nazivamo jednom iteracijom optimizacijskog postupka.

Pojam epohe označava jedan puni prolazak kroz cijeli skup podataka. S obzirom na to da se u gradijentnom spustu gradijent računa nad svim podacima odjednom, svaka epoha se sastoji od točno jedne iteracije.

Trening modela se obično provodi kroz više epoha kako bi se postupno minimizirala vrijednost funkcije gubitka i omogućila konvergencija modela prema optimalnim parametrima.

Jedan od glavnih nedostataka batch gradijentnog spusta je njegova neučinkovitost kod velikih skupova podataka, jer zahtijeva da se u svakoj iteraciji (tj. u svakoj epohi) koristi cijeli skup podataka za računanje gradijenta i ažuriranje parametara.

Zato je u takvim situacijama možda pogodnije koristiti alternativne varijante gradijentnog spusta poput stohastičkog gradijentnog spusta.

2.4.4. Stohastički gradijentni spust

Kod stohastičkog gradijentnog spusta (SGD), gradijent se ne računa nad cijelim skupom podataka već nad jednim ili malim brojem uzoraka (x_i, y_i) , što omogućuje brža ažuriranja i smanjuje memorijske zahtjeve. U jednoj epohi parametri se ažuriraju proizvoljan broj puta ovisno o veličini uzorka. Ažuriranje parametara u SGD-u definira se kao:

$$\theta = \theta - \eta \nabla_{\theta} \mathcal{L}(\theta; x_i, y_i) \quad (2.12)$$

Algoritam SGD-a možemo pojednostavljeno prikazati prema algoritmu 1

Algorithm 1 SGD algoritam

```
for svake epohe do  
  for svakog batcha podataka (ulazi, oznake) do  
    Izračunaj izlaze modela za ulaze u uzorku  
    Izračunaj gubitak između izlaza i oznaka  
    Izračunaj gradijente parametara modela  
    Ažuriraj parametre modela u smjeru negativnog gradijenta  
  end for  
end for
```

3. Suparnički napadi

Suparnički napadi su značajan sigurnosni izazov u suvremenoj umjetnoj inteligenciji. Suparnički primjer je ulazni podatak koji je namjerno modificiran malim, često nezamjetnim promjenama kako bi doveo model dubokog učenja u zabludu i rezultirao pogrešnom klasifikacijom.

Goodfellow i suradnici [8] prvi su otkrili ovu pojavu i time pokazali da se modeli dubokog učenja mogu lako prevariti dodavanjem pažljivo odabranog šuma na ulazne podatke.

Matematički, suparnički primjer x' možemo definirati kao zbroj originalnog podatka x i suparničke perturbacije δ :

$$x' = x + \delta \quad (3.1)$$

Originalni podatak $x \in \mathbb{R}^n$ predstavlja ulazni uzorak za model, dok je perturbacija $\delta \in \mathbb{R}^n$ vektor iste dimenzije koji se dodaje na x s ciljem promjene izlaza modela.

Cilj je pronaći perturbaciju δ takvu da vrijedi $\|\delta\|_p \leq \varepsilon$ (perturbacija je mala u nekoj normi) i $f(x') \neq f(x)$ (model klasificira drugačije).

Iako modeli dubokog učenja često postižu visoke rezultate na testnim skupovima, suparnički napadi pokazuju da ta točnost ne garantira njihovu robusnost u stvarnom svijetu. Mali, namjerno konstruirani poremećaji mogu dovesti do jako loše generalizacijske uspješnosti modela. Ova pojava predstavlja plodno tlo za zlonamjerne manipulacije produkcijskim modelima.

Posebno su zabrinjavajući sigurnosni rizici koje suparnički napadi donose u kritičnim područjima poput autonomnih vozila, medicinske dijagnostike ili sustava za prepozna-

vanje lica. U takvim aplikacijama, zlonamjerni napadi mogu uzrokovati veliku štetu, što naglašava ozbiljnu potrebu za razvojem učinkovitih obrambenih mehanizama.

Proučavanje suparničkih primjera također potiče dublje razumijevanje kako modeli reprezentiraju podatke i pomaže u razvoju robusnijih algoritama.

3.1. Tipovi suparničkih napada

3.1.1. White-box i Black-box napadi

White-box napadi podrazumijevaju da napadač ima potpuni pristup ciljanom modelu, uključujući arhitekturu, parametre i mogućnost računanja gradijenata. Ovakvi napadi su tipično efikasniji od black-box napada jer mogu izravno koristiti informacije o modelu za kreiranje perturbacija.

Black-box napadi simuliraju scenarije gdje napadač nema pristup unutrašnjosti modela, već može samo slati upite i dobivati izlaze. Ovi napadi često koriste surogatne (zamjenske) modele ili algoritme za optimizaciju bez gradijenata.

Najčešće strategije black-box napada uključuju prijenos napada na surogatne modele, iterativno slanje upita ciljnom modelu (query-based napadi) te korištenje genetskih algoritama za učinkovito traženje perturbacija bez pristupa nutrini modela.

3.1.2. Ciljani i neciljani napadi

Neciljani napadi (untargeted attacks) imaju za cilj jednostavno dovesti model do pogrešne klasifikacije, bez obzira u koju će klasu ulaz biti svrstan. Formalno, cilj je pronaći perturbaciju δ takvu da:

$$f(x + \delta) \neq f(x) \quad (3.2)$$

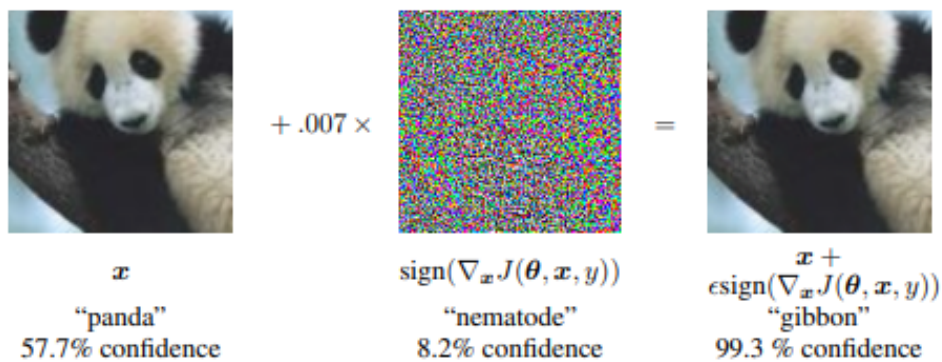
Ciljani napadi (targeted attacks) nastoje dovesti model da klasificira ulaz u specifičnu ciljnu klasu t . Cilj je pronaći perturbaciju δ takvu da vrijedi:

$$f(x + \delta) = t \quad (3.3)$$

Ciljani napadi su općenito složeniji za izvođenje jer moraju zadovoljiti strože uvjete, ali mogu biti opasniji u određenim primjenama gdje je specifična pogrešna klasifikacija posebno štetna.

3.2. Napad FGSM

Napad predznakom gradijenta (eng. Fast Gradient Sign Method, FGSM) je jedna od najjednostavnijih i najšire korištenih metoda za generiranje suparničkih primjera. Razvili su je Goodfellow i suradnici 2015. godine [8] kao odgovor na potrebu za brzom i efikasnom metodom demonstriranja ranjivosti dubokih neuronskih mreža.



Slika 3.1. Generiranje suparničkog primjera na modelu GoogLeNet, za sliku iz skupa podataka ImageNet i perturbacije veličine $\epsilon = 0.007$. Preuzeto iz [8].

Napad FGSM (Fast Gradient Sign Method) koristi linearnu aproksimaciju funkcije gubitka $\mathcal{L}(\theta, x, y)$ oko ulaza x . Pomoću Taylorovog razvoja prvog reda, funkcija gubitka za blago perturbirani ulaz $x + \delta$ može se aproksimirati kao:

$$\mathcal{L}(\theta, x + \delta, y) \approx \mathcal{L}(\theta, x, y) + \delta^T \nabla_x \mathcal{L}(\theta, x, y) \quad (3.4)$$

U prikazanoj jednadžbi 3.4, θ je skup parametara modela, x izvorni ulaz, y odgovarajuća oznaka (labela), a δ mala promjena (perturbacija) nad ulazom. Izraz $\mathcal{L}(\theta, x + \delta, y)$ označava funkciju gubitka izračunatu za perturbirani ulaz $x + \delta$, dok $\mathcal{L}(\theta, x, y)$ predstavlja funkciju gubitka za neperturbirani (izvorni) ulaz. Gradijent $\nabla_x \mathcal{L}(\theta, x, y)$ daje smjer najveće promjene funkcije gubitka s obzirom na ulaz x .

Cilj napada je pronaći malu promjenu δ koja maksimizira gubitak. Budući da je prvi član u jednadžbi 3.4 konstantan, treba maksimizirati:

$$\delta^T \nabla_x \mathcal{L}(\theta, x, y).$$

pod uvjetom da je δ ograničena L_∞ normom: $\|\delta\|_\infty \leq \epsilon$.

Taj izraz je najveći kada su δ i gradijent u istom smjeru, odnosno kada svaka komponenta δ_i ima isti predznak kao odgovarajuća komponenta gradijenta. Optimalna perturbacija δ je stoga:

$$\delta = \epsilon \cdot \text{sign}(\nabla_x \mathcal{L}(\theta, x, y)) \quad (3.5)$$

To znači da svaku komponentu ulaza mijenjamo za najviše ϵ u smjeru u kojem gubitak najbrže raste. Rezultat je suparnički primjer:

$$x_{\text{adv}} = x + \epsilon \cdot \text{sign}(\nabla_x \mathcal{L}(\theta, x, y)) \quad (3.6)$$

Funkcija predznaka sign jednaka je +1 ako je gradijent pozitivan, -1 ako je negativan te 0 kada je gradijent 0.

U slučaju slika, to znači da želimo pomaknuti svaki piksel (i svaki kanal ako imamo sliku u boji) u smjeru u kojem se gubitak najbrže povećava. Taj smjer je određen gradijentom gubitka s obzirom na ulaz $\nabla_x \mathcal{L}(\theta, x, y)$.

Povećanjem vrijednosti ϵ povećava se uspješnost napada no, istovremeno dolazi i do veće vizualne deformacije slike, što može učiniti napad lako uočljivim ljudskom oku. Idealni napadi koriste najmanju moguću vrijednost ϵ koja još uvijek uzrokuje pogrešku u klasifikaciji.

3.2.1. Varijante FGSM-a.

Ciljani i neciljani napad opisani su u 3.1.2, a njihove formulacije u kontekstu FGSM-a su sljedeće:

$$\text{Neciljani napad: } x_{\text{adv}} = x + \epsilon \cdot \text{sign}(\nabla_x \mathcal{L}(\theta, x, y_{\text{true}}))$$

$$\text{Ciljani napad: } x_{\text{adv}} = x - \epsilon \cdot \text{sign}(\nabla_x \mathcal{L}(\theta, x, y_{\text{target}}))$$

Poseban slučaj ciljanog napada je ciljani napad prema najneizglednijem razredu (Least-Likely varijanta). U ovom slučaju, ciljni razred je onaj kojem model inicijalno pridružuje najmanju vjerojatnost:

$$y_{\text{target}} = \arg \min_y P(y | x)$$

3.3. Ostale popularne metode napada

3.3.1. Napad projiciranim gradijentnim spustom (Projected Gradient Descent - PGD)

Napad PGD je iterativna varijanta FGSM-a koju su predložili Madry i suradnici 2018. godine [9]. PGD je značajno poboljšanje u odnosu na jednokratni pristup FGSM-a jer omogućava kreiranje snažnijih suparničkih primjera kroz iterativni proces optimizacije. PGD primjenjuje više koraka gradijentnog spusta s malim koracima pri čemu se perturbirana slika nakon svakog koraka projicira natrag u ϵ -okolinu originalnog primjera kako bi se poštovalo zadano ograničenje perturbacije. ϵ -okolina je skup svih točaka u prostoru koje se nalaze unutar udaljenosti ϵ od originalnog primjera (u nekoj normi).

Iterativno ažuriranje računa se prema sljedećoj jednadžbi, za $t = 0, 1, \dots, T-1$:

$$x_{t+1} = \Pi_S(x_t + \alpha \cdot \text{sign}(\nabla_x \mathcal{L}(\theta, x_t, y))) \quad (3.7)$$

U prikazanoj jednadžbi α je veličina koraka, a Π_S je operator projekcije na vektorski potprostor S oko originalnog primjera:

$$\mathcal{S} = \{x' \mid \|x' - x\|_\infty \leq \epsilon\} \quad (3.8)$$

Ako gornja formula koristi beskonačno normu potprostor $\mathcal{S}$ nazivamo hiperkocka, a ako se koristi L_2 norma onda se radi o hipersferi.

3.3.2. Carlini & Wagner napad

Carlini & Wagner (C&W) napad je jedan od najsnažnijih whitebox napada, koji je predstavljen 2017. godine [10]. Radi se poboljšanoj optimizaciji zbroja norme perturbacije s funkcijom koja modelira uspješnost napada [11].

$$\min_{\delta} \|\delta\|_p + c \cdot g(x + \delta) \quad (3.9)$$

gdje je $\|\delta\|_p$ veličina perturbacije u odabranoj L_p normi, a c je pozitivna konstanta koja određuje ravnotežu između veličine perturbacije i uspješnosti napada. Za razliku od prethodnog rada [11] koji funkciju g izvodi kao unakrsnu entropiju, funkcija g u C&W napadu odgovara razlici između najvećeg logita (osim ciljnog) i logita ciljne klase:

$$g(x') = \max \left(\max_{i \neq t} \{Z(x')_i\} - Z(x')_t, 0 \right) \quad (3.10)$$

U prikazanoj jednadžbi $Z(x')$ je vektor logita (izlaz modela prije primjene softmaxa), a t označava ciljnu klasu. Optimizacijom se pokušava sniziti $g(x')$ na nulu, čime se osigurava da je logit ciljne klase najveći i time $f(x') = t$. Ovakva formulacija bolja je od unakrsne entropije [11] jer ne guši gradijente [10] kriterijske funkcije $J(\delta)$.

C&W napad je poznat po visokoj uspješnosti i sposobnosti zaobilaženja mnogih obrambenih mehanizama, uključujući metode detekcije i regularizacije. Najčešće se koristi u L_2 normi, ali postoje i varijante za L_0 i L_∞ norme.

3.3.3. One Pixel Attack

One Pixel Attack je black-box napad koji pokazuje da je moguće prevariti modele mijenjanjem samo jednog (ili vrlo malog broja) piksela na slici. [12]

Ova metoda koristi Differential Evolution (DE) algoritam za pronalaženje optimalnog piksela (ili više njih) i njegove nove RGB vrijednosti (ili intenziteta ako je slika crno bijela).

Svaki kandidat u populaciji DE algoritma predstavlja se kao vektor:

$$x = (x_1, y_1, r_1, g_1, b_1, \dots, x_n, y_n, r_n, g_n, b_n)$$

, gdje (x_i, y_i) označavaju koordinate i -tog piksela, a (r_i, g_i, b_i) njegove nove RGB vrijednosti za n modificiranih piksela.

Vrijednost n odabire se unaprijed kao mali cijeli broj, obično 1, ovisno o kompromisu između neprimjetnosti izmjene i učinkovitosti napada.

DE algoritam iterativno poboljšava populaciju kroz tri glavne operacije:

1. **Mutacija:** Za svaki vektor x_i stvara se mutant:

$$v_i = x_{r1} + F \cdot (x_{r2} - x_{r3})$$

gdje su $r1, r2, r3$ nasumično odabrani različiti indeksi, a F je faktor mutacije.

2. **Križanje:** Stvara se trial vektor u_i kombiniranjem x_i i v_i prema vjerojatnosti križanja CR .

3. **Selekcija:** Zadržava se bolje rješenje između x_i i u_i na osnovu vrijednosti funkcije dobrote (fitness).

One Pixel Attack demonstrira ekstremnu ranjivost dubokih neuronskih mreža na minimalne perturbacije (čak i jedna modificirana vrijednost piksela može dovesti do pogrešne klasifikacije) za čije generiranje nisu potrebni gradijenti modela.

3.4. Obrane od suparničkih napada

Razvoj suparničkih napada potaknuo je istraživanje različitih obrambenih mehanizama. Glavne kategorije obrane uključuju suparničko treniranje, maskiranje gradijenata, obrambenu destilaciju i certificirane obrane.

Suparničko treniranje

Suparničko treniranje proširuje skup podataka za treniranje suparničkim primjerima kako bi model naučio ispravno prepoznati suparničke primjere. [9] Ovakvo učenje trenutno je jedna od najučinkovitijih obrana od postojećih suparničkih napada.

Maskiranje gradijenata

Maskiranje gradijenata nastoji prikriti gradijente modela što otežava kreiranje suparničkih primjera. Uključuje tehnike poput razbijenih gradijenata, stohastičkih gradijenata i eksplodirajućih/nestajućih gradijenata [13]. Međutim, napredni adaptivni napadi često mogu zaobići ove obrane.

Certificirane obrane

Certificirane obrane pružaju teorijske garancije o robusnosti modela unutar zadane ϵ -okoline. Pružaju matematičke dokaze sigurnosti, ali s kompromisima u točnosti i računalnoj složenosti [14].

4. Implementacija i rezultati

U praktičnom dijelu rada napravljeno je sljedeće:

- implementacija SGD algoritma (1) za optimizaciju potpuno povezanog modela za klasifikaciju skupa podataka MNIST,
- implementacija napada FGSM,
- primjena ciljanog i neciljanog napada na modele za klasifikaciju skupova podataka MNIST i CIFAR100.

4.1. Razvojno okruženje

4.1.1. Python

Za implementaciju sustava korišten je programski jezik Python. Python je odabran zbog svoje široke primjene u području strojnog i dubokog učenja, kao i zbog dostupnosti brojnih specijaliziranih biblioteka koje olakšavaju razvoj takvih sustava, poput biblioteka NumPy za numeričke operacije i Matplotlib za prikaz rezultata i analiza. Posebno je važno istaknuti biblioteku PyTorch, koja je korištena kao temelj za implementaciju modela i izvođenje naprednih računskih operacija, a detaljnije je opisana u sljedećem poglavlju.

4.1.2. PyTorch

PyTorch je korišten kao programsko okruženje za implementaciju i treniranje modela. Posebno korisna podrška je automatsko računanje gradijenata, što je u ovom radu iskorišteno za optimizaciju parametara modela i generiranje suparničkih perturbacija. Općenito, PyTorch je omogućio jednostavno definiranje strukture dubokih modela, uprav-

ljanje podacima i izvođenje procesa učenja.

Za implementaciju su bile važne sljedeće komponente: `torch.nn` za definiranje arhitekture modela, `torch.optim` za optimizacijske algoritme, `torchvision` za dohvata i obradu slikovnih skupova podataka i osnovne transformacije.

4.1.3. Skup podataka MNIST

MNIST (Modified National Institute of Standards and Technology) je skup podataka koji sadrži 70,000 slika rukom pisanih znamenaka (0-9), od kojih je 60,000 skup za učenje, a 10,000 skup za testiranje. Skup za učenje koristimo za optimizaciju parametara, a skup za testiranje za ocjenjivanje generalizacijske uspješnosti modela i generiranje suparničkih primjera.

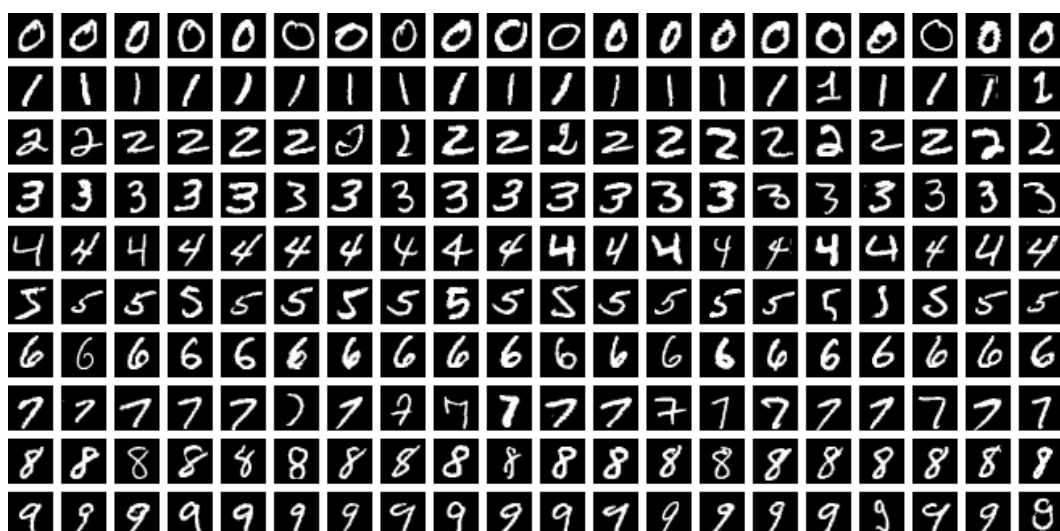
Svaka slika je jednokanalna (u nijansama sive), dimenzija 28×28 piksela, što daje ukupno 784 značajke po slici. Vrijednosti piksela inicijalno su cjelobrojne i kreću se u rasponu od 0 do 255, pri čemu 0 predstavlja crnu, a 255 bijelu boju. Takve "sirove" vrijednosti piksela rijetko se koriste u praksi jer nisu pogodne za većinu optimizacijskih algoritama i mogu usporiti ili otežati učenje modela.

U ovom radu slike su najprije skalirane u raspon $[0, 1]$ (transformacija `ToTensor()`), a zatim normalizirane (`Normalize(std, mean)`) s parametrima srednje vrijednosti 0.1307 i standardne devijacije 0.3081. Ove vrijednosti odgovaraju statističkim parametrima cijelog MNIST skupa i često se koriste u literaturi kako bi se ubrzalo i stabiliziralo učenje modela.

Transformacije su definirane korištenjem modula `transforms` biblioteke `torchvision`, a zatim primijenjene pri učitavanju podataka pomoću funkcije `torchvision.datasets.MNIST()`. Organizacija podataka u mini-grupe i njihovo učinkovito posluživanje modelu tijekom učenja i evaluacije ostvarena je korištenjem razreda `DataLoader` iz modula `torch.utils.data`.

4.1.4. Skup podataka CIFAR-100

CIFAR-100 je skup podataka koji se sastoji od 60 000 slika u boji dimenzija 32×32 piksela, pri čemu je svaka slika predstavljena s tri kanala (RGB).



Slika 4.1. primejri slika iz skupa MNIST. Preuzeto iz [15].

Vrijednosti piksela inicijalno se nalaze u rasponu od 0 do 255, no kao i kod MNIST skupa, slike su skalirane i normalizirane kako bi se omogućio stabilniji i učinkovitiji rad modela. U ovom radu korištene su srednje vrijednosti (0.5071, 0.4867, 0.4408) i standardne devijacije (0.2675, 0.2565, 0.2761) po kanalima crvena–zelena–plava (RGB), koje su uobičajene u literaturi za obradu CIFAR-100 skupa.

Slike su podijeljene u 100 različitih klasa tako da svaka klasa ima po 600 slika, od čega je 500 namijenjeno za učenje, a 100 za testiranje. U usporedbi s poznatijim skupom CIFAR-10, koji sadrži samo 10 šire definiranih kategorija, CIFAR-100 predstavlja proširenje s finijom kategorizacijom. Klase u CIFAR-100 organizirane su hijerarhijski: postoji 20 "superklasa" (coarse label), od kojih svaka sadrži po 5 "podklasa" (fine label). Međutim, u ovom radu koristi se standardna podjela s 100 osnovnih klasa, bez korištenja hijerarhijske strukture.

Iako CIFAR-100 sadrži manje slika od MNIST-a, slike su u boji i sadrže znatno veći raspon objekata i vizualnih razlika unutar 100 klasa. To otežava proces učenja, osobito kod dubljih modela koji su potrebni za postizanje zadovoljavajuće točnosti. Kako bismo se mogli usredotočiti na ispitivanje robusnosti modela i implementaciju suparničkih napada, odlučili smo koristiti unaprijed istrenirani model ResNet-20 za CIFAR-100, preuzet putem PyTorch modula `torch.hub`.

S obzirom da nismo provodili treniranje modela, a napad se provodi s testnim primjermima, skup za učenje nam nije bio potreban, već se koristio samo skup za testiranje

veličine 10,000 slika.

4.2. Treniranje modela na skupu podataka MNIST

Za klasifikaciju rukom pisanih znamenki iz MNIST skupa korišten je jednostavni potpuno povezani model treniran algoritmom stohastičkog gradijentnog spusta (SGD) koji je opisan u ranijim poglavljima.

4.2.1. Arhitektura mreže

Za klasifikaciju znamenaka iz MNIST skupa podataka korišten je jednostavan potpuno povezani model.

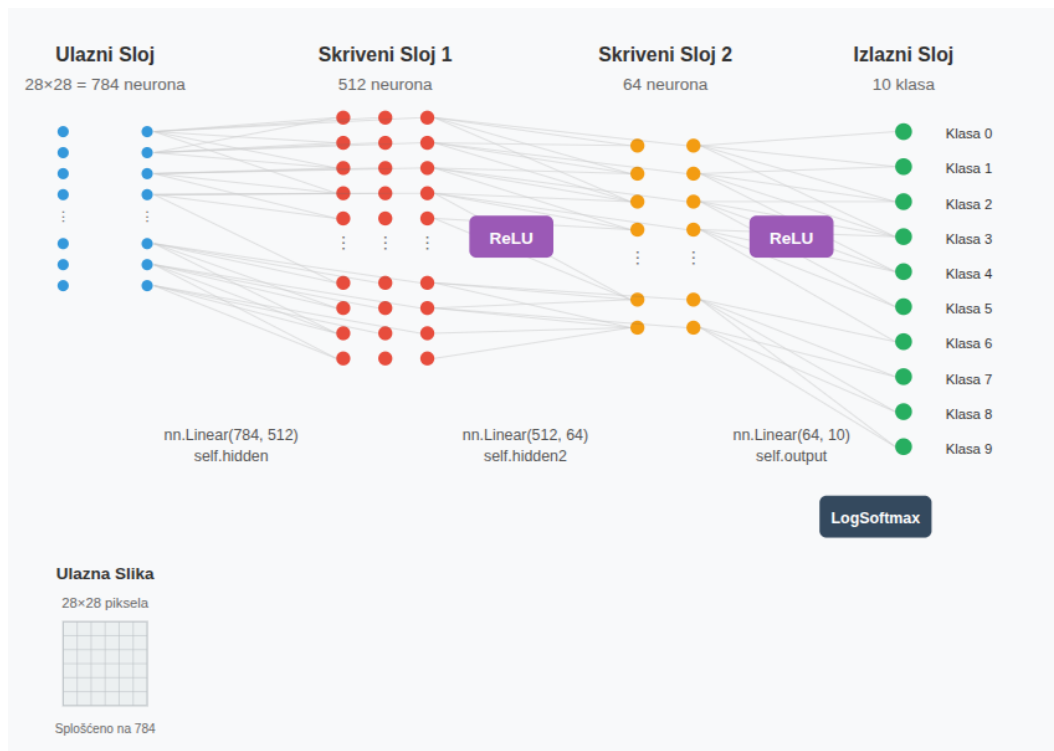
Ulazni sloj prima sliku dimenzije 28×28 piksela koja se prije ulaska u mrežu transformira u vektor duljine 784. Prvi skriveni sloj ima 512 neurona, dok drugi skriveni sloj ima 64 neurona. Izlazni sloj sastoji se od 10 neurona, koji odgovaraju klasama znamenki 0–9.

Oba skrivena sloja koriste ReLU (engl. Rectified Linear Unit) aktivacijsku funkciju 2.2, a na izlazu modela koristi se funkcija LogSoftmax, koja daje logaritamsku vjerojatnost svake klase i kompatibilna je s funkcijom gubitka negativne log-izglednosti 2.3.3. (NLLLoss) korištenom pri učenju modela.

Programsko ostvarenje takve arhitekture postignuto je sljedećim kodom:

```
class ImageClassifier(nn.Module):
    def __init__(self):
        super().__init__()
        self.hidden = nn.Linear(28*28, 512)
        self.hidden2 = nn.Linear(512, 64)
        self.output = nn.Linear(64, 10)
        self.softmax = nn.LogSoftmax(dim=1)

    def forward(self, x):
        x = self.hidden(x)
        x = F.relu(x)
        x = self.hidden2(x)
        x = F.relu(x)
```



Slika 4.2. Arhitektura modela korištenog za klasifikaciju MNIST skupa podataka. Generirano s Claude.ai

```
x = self.output(x)
x = self.softmax(x)
return x
```

U cilju brze optimizacije, odabrana arhitektura relativno je plitka te ima relativno maleni broj parametara. Također, u području računalnog vida, odabir potpuno povezanog modela u pravilu rezultira slabijim rezultatima i lošijom generalizacijom od kompleksnijih arhitektura poput konvolucijskih modela. Međutim, za jednostavan skup poput MNIST-a, ovakav model često pokazuje zadovoljavajuće performanse, osobito kada se pravilno skaliraju i normaliziraju ulazne slike.

4.2.2. Optimizacija parametara

Model optimiramo algoritmom stohastičkog gradijentnog spusta (SGD), koji je opisan u odjeljku 2.4.4. Za funkciju gubitka odabrana je negativna log-izglednost (2.3.3.), koja se često koristi u kombinaciji s LogSoftmax na izlazu klasifikacijskih modela.

Hiper-parametri treniranja bili su sljedeći:

- Broj epoha: 10
- Veličina grupe (batch size): 64
- Stopa učenja (learning rate): 0.01

Odsječak izvornog koda s petljom učenja prikazan je u nastavku. Referenca optimizer postavljena je na `optim.SGD(imageClassifier.parameters(), lr=0.01)`, a referenca criterion postavljena je na `criterion = nn.NLLLoss()`.

```
for epoch in range(epochs):
    #ucimo na skupu podataka za treniranje
    running_loss = 0
    for images, labels in trainloader:
        images = images.view(images.shape[0], -1) # pretvori u
            vektor
        optimizer.zero_grad() # ponisti gradijente
        probs = imageClassifier(images) # Forward pass
        loss = criterion(probs, labels)
        loss.backward() # Backward pass
        optimizer.step() # azuriraj parametre
        running_loss += loss.item()

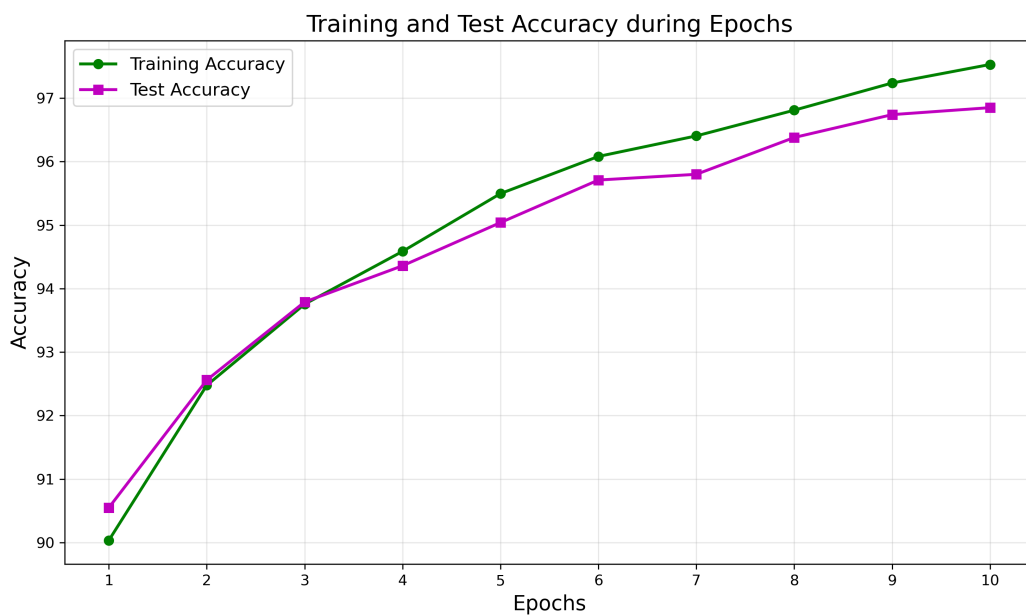
    #nakon svake epohe evaluiramo na testnom skupu podataka
    accuracy_test = calculate_accuracy(imageClassifier, testloader
    )
```

Za svaku epohu model se trenirao na cijelom skupu za učenje, a nakon svake epohe evaluiran je i na skupu za testiranje. Praćene su metrike točnosti i funkcije gubitka na skupu za učenje, i na skupu za testiranje te su opisane sljedećim grafovima:

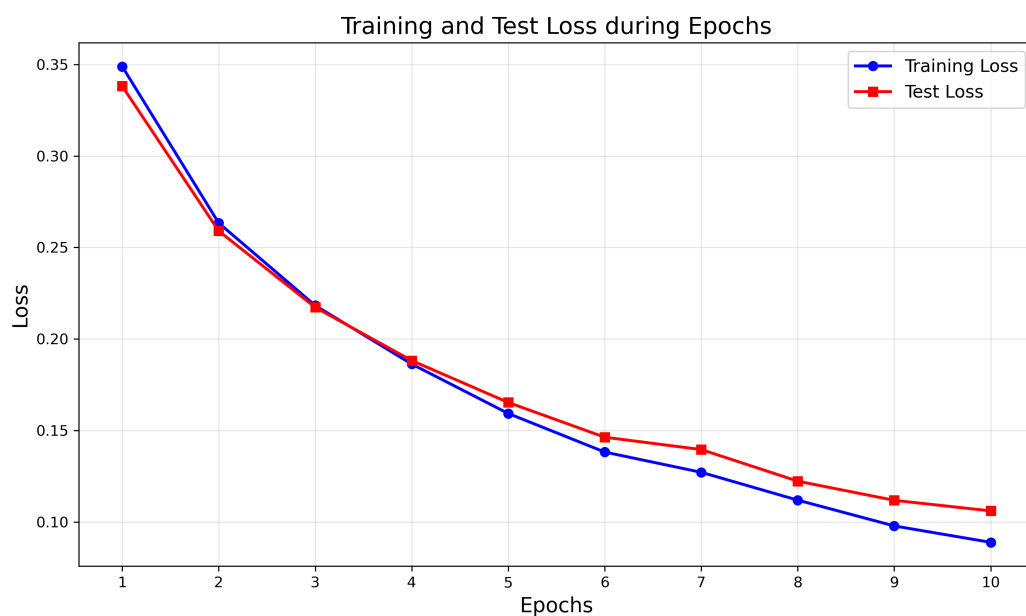
Već nakon 10 epoha točnost na testnom skupu je prešla 95%, što je dovoljno dobar rezultat za naše potrebe te zbog toga nismo provodili daljnje treniranje.

Prikazani grafovi sugeriraju uspješno treniranje modela. Graf točnosti (Slika 4.3.) pokazuje kontinuirani rast performansi tijekom 10 epoha, a graf funkcije gubitka (Slika 4.4.) konzistentno opadanje tijekom epoha.

Posebno je važno primijetiti relativno malu razliku između krivulja točnosti za uče-



Slika 4.3. Točnost modela tijekom epoha na skupu za učenje i testiranje.



Slika 4.4. Vrijednost funkcije gubitka modela tijekom epoha na skupu za učenje i testiranje.

nje i testiranje, što ukazuje na dobru generalizaciju modela. Ovakav obrazac sugerira da model ne pokazuje znakove značajne prenaučivosti, što bi se inače manifestiralo velikim razmakom između krivulja gdje bi točnost na skupu za učenje bila značajno viša od točnosti na testnom skupu. Bliskost krivulja gubitka za učenje i testiranje ponovno potvrđuje dobru generalizaciju modela.

Trend opadanja funkcije gubitka i rasta točnosti u zadnjim epohama pokazuje da model i dalje uči, što sugerira da bi dodatno treniranje moglo rezultirati daljnjim poboljšanjima. Međutim, s obzirom na postignute rezultate i rizik od prenaučavanja pri dugotrajnom treniranju, 10 epoha predstavlja razuman kompromis između performansi i efikasnosti treniranja.

4.3. Model za klasifikaciju CIFAR-100 skupa podataka

Za CIFAR-100 preuzet je prednaučeni model resnet-20 čija je arhitektura opisana u 2.2.1.

Model je dobiven korištenjem PyTorch funkcionalnosti `torch.hub` koja omogućuje jednostavan pristup prednaučenim modelima iz različitih repozitorija. Ovakav pristup eliminira potrebu za dugotrajnim treniranjem modela i osigurava korištenje optimiziranih parametara. Repozitorij `chenyafo/pytorch-cifar-models` sadrži kolekciju modela specifično optimiziranih za CIFAR skupove podataka.

Model je preuzet na sljedeći način:

```
model = torch.hub.load('chenyafo/pytorch-cifar-models', '
    cifar100_resnet20', pretrained=True)
torch.save(model.state_dict(), 'cifar100_model.pth')
```

Isprobane su performanse na CIFAR-100 skupu za učenje i model postiže 68.83% točnosti i prosječni gubitak 1.2253. Kao funkcija gubitka koristila se unakrsna entropija 2.3.2. Ove performanse su u skladu s očekivanjima za ResNet-20 arhitekturu na CIFAR-100 skupu, gdje se uobičajeno postižu točnosti između 65-70% bez dodatnih tehnika optimizacije. U kontekstu dubljih modela, ResNet-56 ili ResNet-110 mogu postići nešto bolje rezultate (oko 72-75%), ali uz značajno povećanje računalnih zahtjeva. CIFAR-100 skup podataka općenito predstavlja izazov za klasifikacijske modele zbog fine granularnosti svojih 100 klasa, gdje mnoge klase dijele slične vizualne karakteristike.

ResNet-20 model odabran je prvenstveno zbog usredotočenosti istraživanja na robusnost i suparničke napade, a ne na postizanje maksimalnih performansi klasifikacije. Unatoč tome, zahvaljujući svojoj arhitekturi od 20 slojeva i rezidualnim vezama, model svejedno postiže solidne rezultate u klasifikaciji.

4.4. FGSM implementacija

FGSM napad (3.2.) testiran je na oba implementirana modela - potpuno povezanom modelu za MNIST klasifikaciju i ResNet-20 modelu za CIFAR-100 klasifikaciju. Implementacija je provedena kroz modularan pristup koji omogućuje primjenu napada na različite skupove podataka i arhitekture modela.

4.4.1. Odvajanje logike modela i napada

Kako bi se omogućila jednostavna i fleksibilna primjena FGSM napada na različite modele i skupove podataka korišten je oblikovni obrazac strategija. Ovaj pristup omogućuje da se sve specifične operacije vezane uz arhitekturu modela i oblik podataka enkapsuliraju unutar zasebnih klasa (strategija), čime se glavni napad može provesti generički, bez obzira na konkretni model ili skup. Konkretno, u ovom radu, razlike između MNIST i CIFAR-100 skupova podataka te arhitektura potpuno povezanog modela i modela resnet-20 transparentno su integrane bez izmjene osnovne logike napada.

Svaka strategija mora implementirati sljedeće sučelje:

```
class Strategy:
    def get_test_loader(self)          # DataLoader s transformacijama
        i batch_size=1
    def load_model(self)               # ucitavanje i postavka modela
    def prepare_input(self, images)    # priprema ulaza za model
    def get_predictions(self, outputs) # interpretacija izlaza modela
    def calculate_loss(self, outputs, labels, target) # funkcija
        gubitka
    def normalize(self, batch)         # normalizacija podataka
    def denormalize(self, batch)      # denormalizacija podataka
    def get_num_classes(self)         # broj klasa
    def get_class_names(self)         # nazivi klasa
```

4.4.2. Razlike između MNIST i CIFAR-100 strategije

Osim očekivanih razlika u učitavanju modela i testnog skupa, glavne razlike između razreda MNIST_strategy i CIFAR100_strategy odnose se na način obrade podataka i definiciju ulaza/izlaza modela. Te razlike sažete su u nastavku:

MNIST strategija

- **normalize()/denormalize()**: jednokanalne slike, $\mu = 0.1307$, $\sigma = 0.3081$.
- **prepare_input()**: 2D slika se pretvara u 1D vektor.
- **get_predictions()**: izlaz iz `LogSoftmax`, vjerojatnosti se dobivaju kao `exp(outputs)`.
- **calculate_loss()**: koristi se negativna logaritamska izglednost `NLLoss` 2.3.3.
- **load_model()**: stvara objekt `ImageClassifier()` i učitava parametre iz `last_trained.pth`.
- **get_test_loader()**: vraća `DataLoader` s transformacijama skaliranja i normalizacije specifičnim za MNIST.

CIFAR-100 strategija

- **normalize()/denormalize()**: trokanalne RGB slike, $\mu = (0.5071, 0.4867, 0.4408)$, $\sigma = (0.2675, 0.2565, 0.2761)$.
- **prepare_input()**: zadržava 2D strukturu slike (nije potrebna konverzija u vektor).
- **get_predictions()**: izlaz je logit, vjerojatnosti se dobivaju funkcijom `softmax`.
- **calculate_loss()**: koristi se unakrsna entropija `CrossEntropyLoss` 2.3.2.
- **load_model()**: model se učitava preko `torch.hub.load(...)` i parametri se učitavaju iz `cifar100_model.pth`.
- **get_test_loader()** vraća `DataLoader` s transformacijama skaliranja i normalizacije specifičnim za CIFAR-100.

4.4.3. Logika napada

Funkcija `runFGSM()` implementira glavni algoritam napada kroz iteraciju preko testnog skupa podataka. Za svaku sliku, napad se izvodi samo ako je model inicijalno ispravno klasificirao sliku, što osigurava smislenu evaluaciju degradacije performansi.

```
def runFGSM(strategy, epsilon, target=None):  
  
    model = strategy.load_model()
```

```

test_loader = strategy.get_test_loader()
#test_loader mora imati batch_size 1
test_loader = DataLoader(test_loader.dataset, batch_size=1,
                          shuffle=False)

successes = 0
total = 0
adv_examples=[]

for images, labels in test_loader:
    images.requires_grad = True
    input = strategy.prepare_input(images)
    outputs = model(input)

    probs, predicted = strategy.get_predictions(outputs)
    #radimo napad samo ako se slika inicijalno dobro klasificira
    if predicted.item() == labels.item():

        total += 1
        loss = strategy.calculate_loss(outputs, labels, target)

        model.zero_grad()#ponisti prosle gradijente
        loss.backward()#izracunaj nove gradijente
        images_grad = images.grad.data

        adv_images = fgsm(images, epsilon, images_grad)
        adv_input = strategy.prepare_input(adv_images)

        #breakpoint()
        with torch.no_grad():
            new_outputs = model(adv_input)
            new_probs, new_predicted = strategy.get_predictions(
                new_outputs)

            #uspjeh <=> nontargeted i nije dobra klasifikacija ili
            targeted i klasifikacija je target
            if (target == None and new_predicted.item() != labels.item
                ()) or (target!=None and new_predicted.item() ==

```

```

target[labels.item()]):
    #napad je bio uspjesan
    successes += 1
    if len(adv_examples) < 3:
        images_denorm = strategy.denormalize(images)
        adv_images_denorm = strategy.denormalize(
            adv_images)
        adv_ex = [adv_images_denorm.squeeze().detach().
            numpy(), new_probs.item()]
        initial_ex = [images_denorm.squeeze().detach().
            numpy(), probs.item()]
        adv_examples.append((adv_ex, initial_ex,
            new_predicted.item(), labels.item()))

return successes, total, adv_examples

```

Funkcija `fgsm()` generira suparnički primjer tako da računa perturbaciju koristeći gradijente i dodaje je ulaznom podatku:

```

def fgsm(image, epsilon, data_grad):
    sign_data_grad = data_grad.sign() # .sign() vraca 1 ako je
        gradijent pozitivan a -1 ako je negativan
    #dodajemo epsilon u smjeru gradijenta
    new_image = image + epsilon * sign_data_grad
    return new_image

```

4.4.4. Ciljani i neciljani napad

Funkcija `runFGSM` omogućava provedbu ciljanog i ne ciljanog napada ovisno o tome kako je postavljen parametar `target`. Ukoliko je `target = None`, provodi se neciljani napad, a ako se želi provesti ciljani napad `target` mora biti vektor koji određuje za svaku klasu koja klasa mora biti rezultat klasifikacije suparničkog primjera.

Pri računanju gubitka, funkciji `strategy.calculate_loss()` se kao parametar predaje `target` i ovisno o tome se računa gubitak. Primjer za `MNIST_strategy`:

```

def calculate_loss(self, outputs, labels, target=None):
    if target is None:
        loss = F.nll_loss(outputs, labels)

```

```

else:
    loss = outputs[0, target[labels.item()]]
return loss

```

Također ovisno o parametru target interpretiramo kada ćemo napad smatrati uspješnim. Isječak izvornog koda iz runFGSM() funkcije:

```

# uspjeh <=> nontargeted i nije dobra klasifikacija ili targeted
# i klasifikacija je target
if ( target == None and new_predicted.item () != labels.item() )
or ( target != None and new_predicted.item ()
==target [ labels.item () ]) :
    # napad je bio uspjesan

```

Važno je napomenuti da su modeli trenirani na normaliziranim podacima te se gradijenti također računaju s obzirom na normalizirane vrijednosti. Zato ima najviše smisla perturbaciju dodati normaliziranim podacima jer je na temelju tih podataka perturbacija i izračunata.

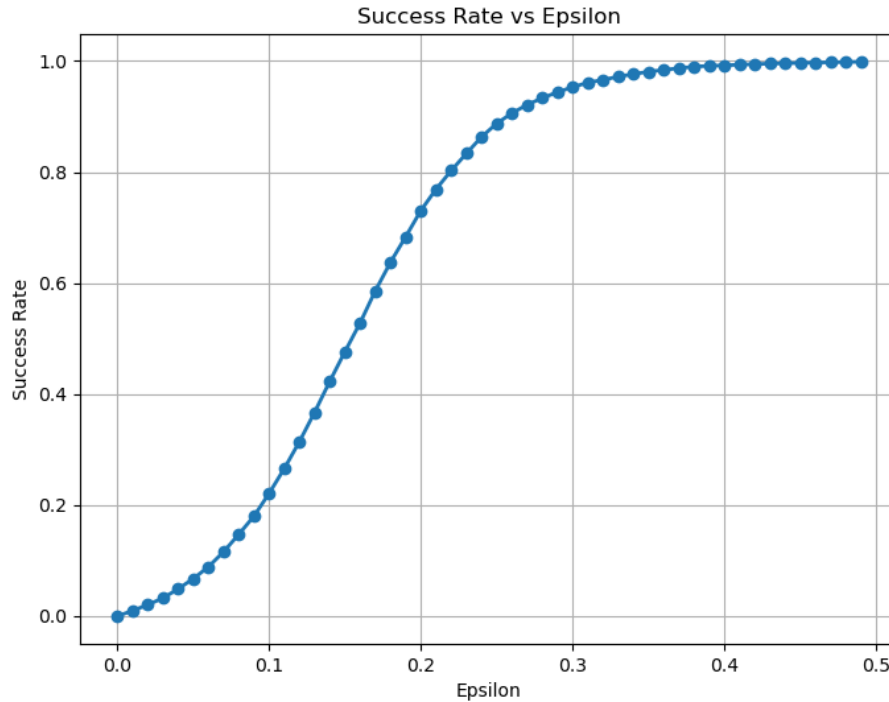
Važno je napomenuti i to da denormalizirana vrijednost pixela nije isto kao originalna vrijednost koja je u rasponu 0-255 već je skalirana e na [0,1] interval jer se normalizacija provodila nad skaliranim slikama. To je bitno imati u vidu pri vizualizaciji slika.

4.5. Rezultati za potpuno povezani model i MNIST

4.5.1. Neciljani napad

Isprobali smo neciljani FGSM napad s različitim normama perturbacije. Slika 4.5. prikazuje uspješnost napada na testnom skupu podatka u ovisnosti o ϵ . Vidimo da se za dovoljno veliki ϵ može postići da napad bude uspješan za svaku sliku iz testnog skupa podataka. Međutim, tada je vrlo očito dodavanje perturbacije na suparničkom primjeru. Kako bismo pokazali vidljivost perturbacije za različite razine učinkovitosti napada, vizualizirat ćemo nekoliko primjera za različite epsilone. Prikazat ćemo originalne slike, iste te slike nakon dodavanja suparničke perturbacije, i vizualizaciju predznaka gradijenata (`data_grads.sign()`) koja pokazuje gdje se dodaje $+\epsilon$ (bijeli pikseli) ili $-\epsilon$ (crni pikseli). Primjerice za $\epsilon = 0.4$, uspješnost napada je skoro 100%, ali tada prosječna i neupućena osoba može uočiti da se ne radi o originalnoj slici 4.6.

Slika 4.7. prikazuje primjere za $\epsilon = 0.2$, kada je uspješnost napada 73%. Možemo primjetiti da je sada perturbacija (gotovo) neprimjetna ljudskom oku, što je svakako poželjno svojstvo ako se stavimo u ulogu napadača. Zato bismo prije mogli očekivati napade s nešto manjim perturbacijama kako bi se postigao kompromis između učinkovitosti i neprimjetnosti suparničkih perturbacija.



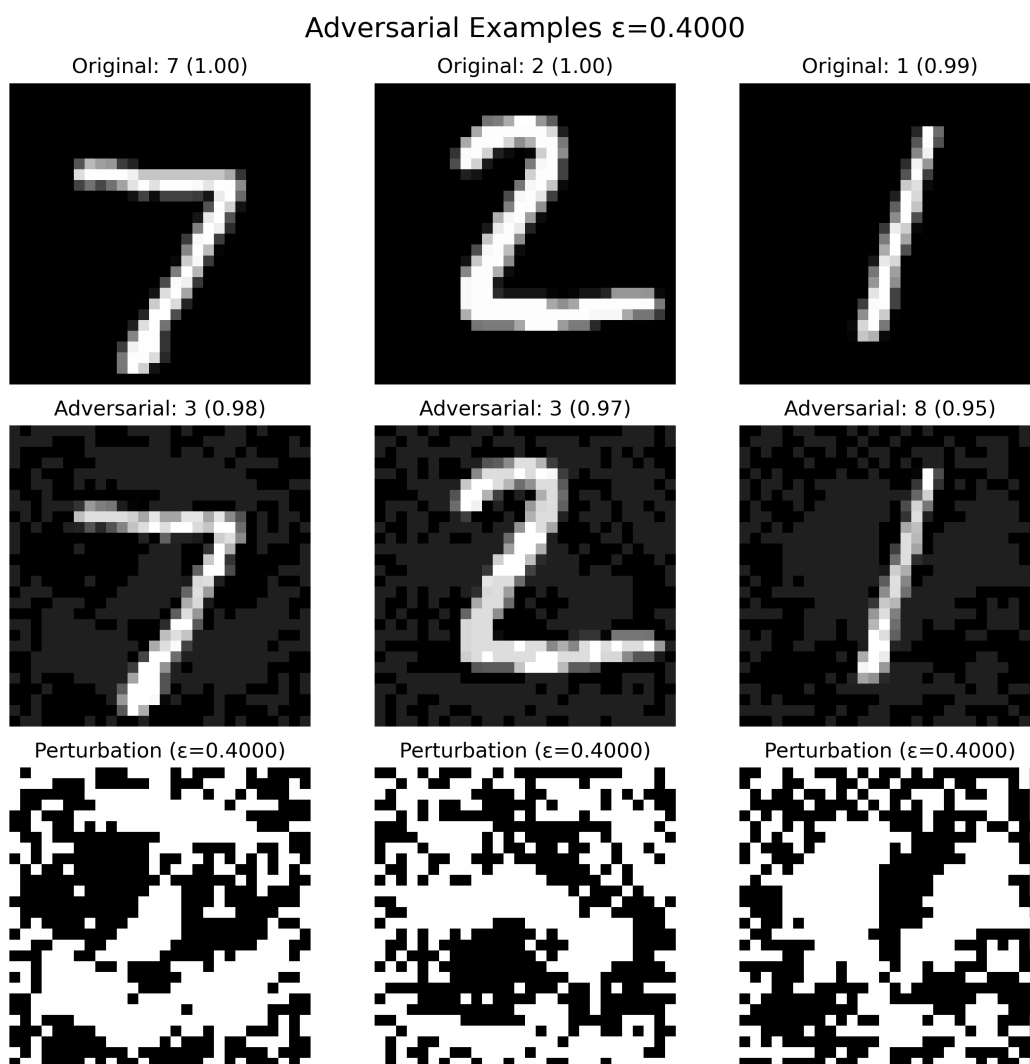
Slika 4.5. Uspješnost neciljanog napada na ispitnom podskupu skupa MNIST.

4.5.2. Ciljani napad

U provedbi ciljanog napada parametar `target` je `target=[1,2,3,4,5,6,7,8,9,0]`. To znači da je cilj postići da se primjer koji se inicijalno klasificira u klasu t , nakon dodavanja neprijateljske perturbacije klasificira u klasu $t + 1$. Dakle, nula treba postati jedan, jedan treba postati 2 i tako dalje.

Ponovo je napravljen graf kao i za neciljani napad. Slika 4.8. prikazuje ovisnost uspješnosti napada o ϵ -u.

Iz grafa je vidljivo da je teže postići uspješan ciljani napad nego uspješan neciljani napad jer se kod ciljanog napada za iste vrijednosti ϵ dobiva slabija uspješnost. To je i očekivano ponašanje, jer je lakše postići klasifikaciju u bilo koju "krivu" klasu nego

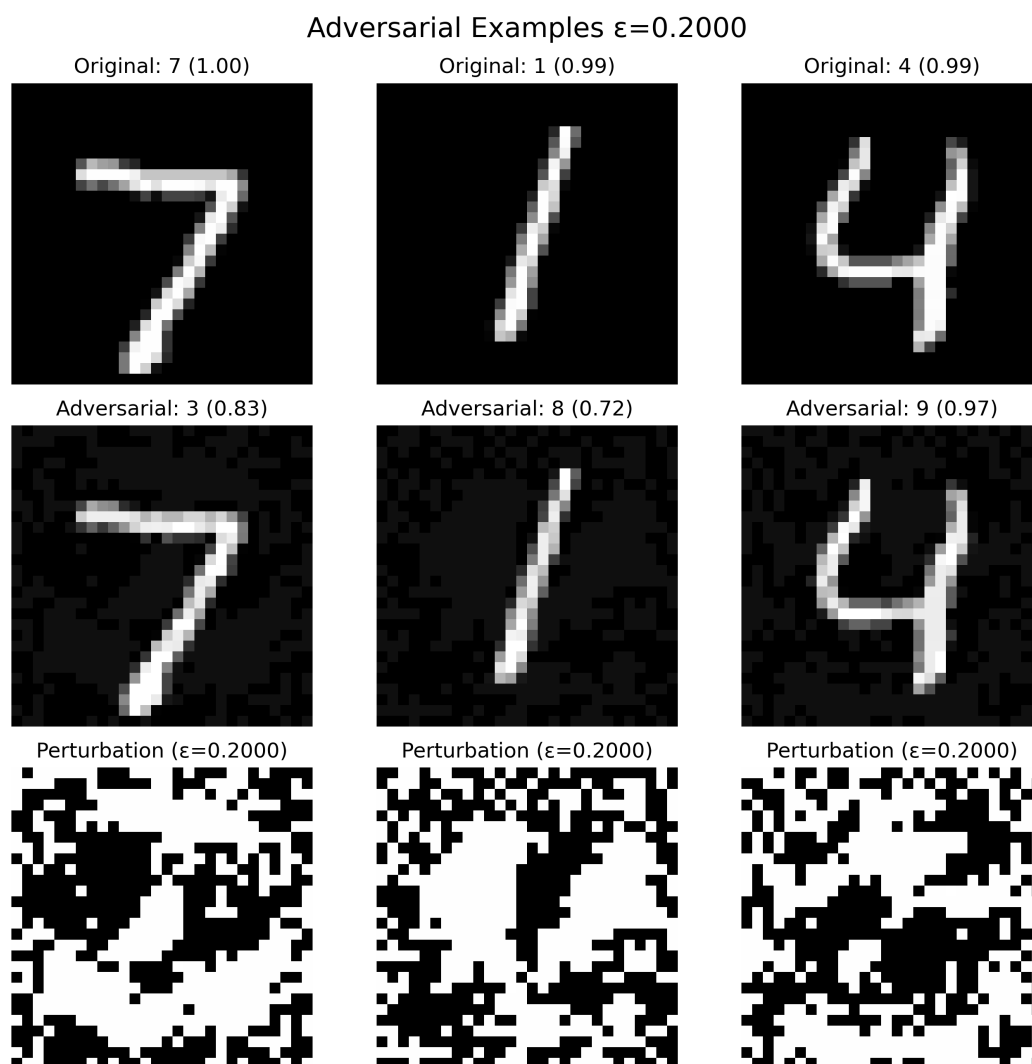


Slika 4.6. Suparnički primjeri: MNIST, neciljani napad, $\epsilon = 0.4$.

klasifikaciju u točno određenu "krivu" klasu. Primjerice, za $\epsilon = 0.4$ koji je u neciljanom napadu davao uspješnost preko 99%, sada daje tek 68.9%.

Kada ϵ postane prevelik, generirane perturbacije značajno izobličuju sliku, što vodi do vidljivih deformacija i gubi se smisao napada. Štoviše, teorijski temelj FGSM-a (linearizacija pomoću Taylorovog razvoja 3.4) pretpostavlja da su perturbacije dovoljno male da vrijedi aproksimacija Taylorovim razvojem prvog reda. Stoga korištenje velikih vrijednosti ϵ nije samo praktički neprikladno, već i protivno teorijskoj osnovi metode. Zbog toga nismo provodili eksperimente za ϵ veći od 0.5.

Slika 4.9. prikazuje suparničke primjere za $\epsilon = 0.5$ koji je odabran kao gornja granica mjere perturbacije pri provedbi napada.



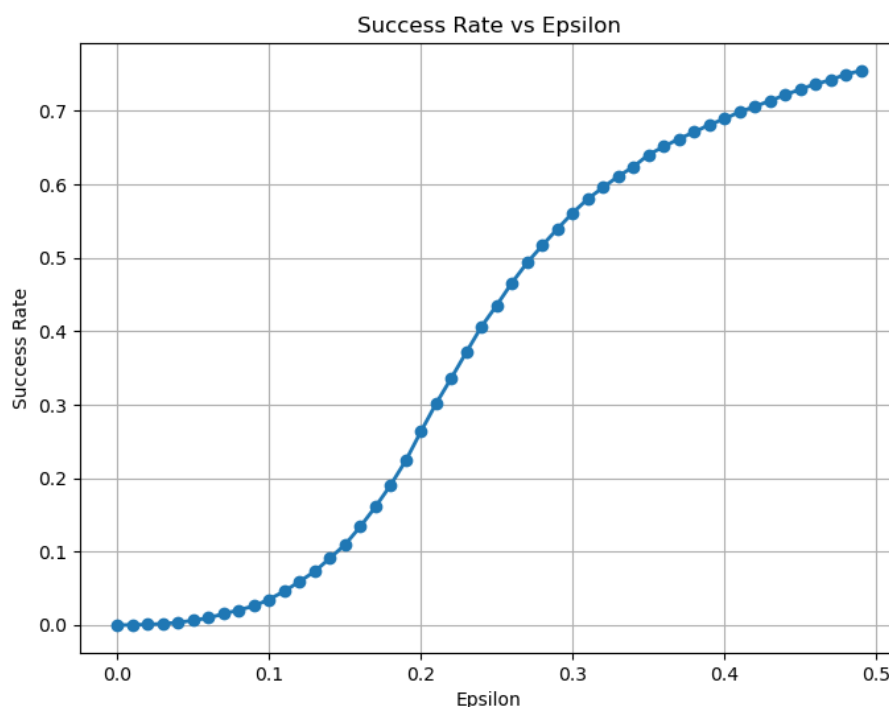
Slika 4.7. Suparnički primjeri: MNIST, neciljani napad, $\epsilon = 0.2$.

4.6. Rezultati za resnet20 model i CIFAR-100

4.6.1. Neciljani napad

Neciljani napad je proveden isto kao i za MNIST te ponovo prikazujemo rezultate na isti način kao i za MNIST. Slika 4.10. prikazuje uspješnost neciljanog napada u ovisnosti o vrijednostima ϵ za model resnet-20 i skup podataka CIFAR100.

Vidimo da uspješnost vrlo brzo raste na intervalu $[0, 0.05]$ te kasnije polako konvergira. Za vrlo male epsilone mozemo primjetiti prilično veliku uspješnost. Već vrijednost $\epsilon = 0.05$ postiže se preko 90% uspješnosti na testnom skupu podatka, a perturbacija je neprimjetna ljudskom oku. Slika 4.11. prikazuje suparničke primjere za tu vrijednost.



Slika 4.8. Uspješnost ciljanog napada na ispitnom podskupu skupa MNIST.

4.6.2. Ciljani napad

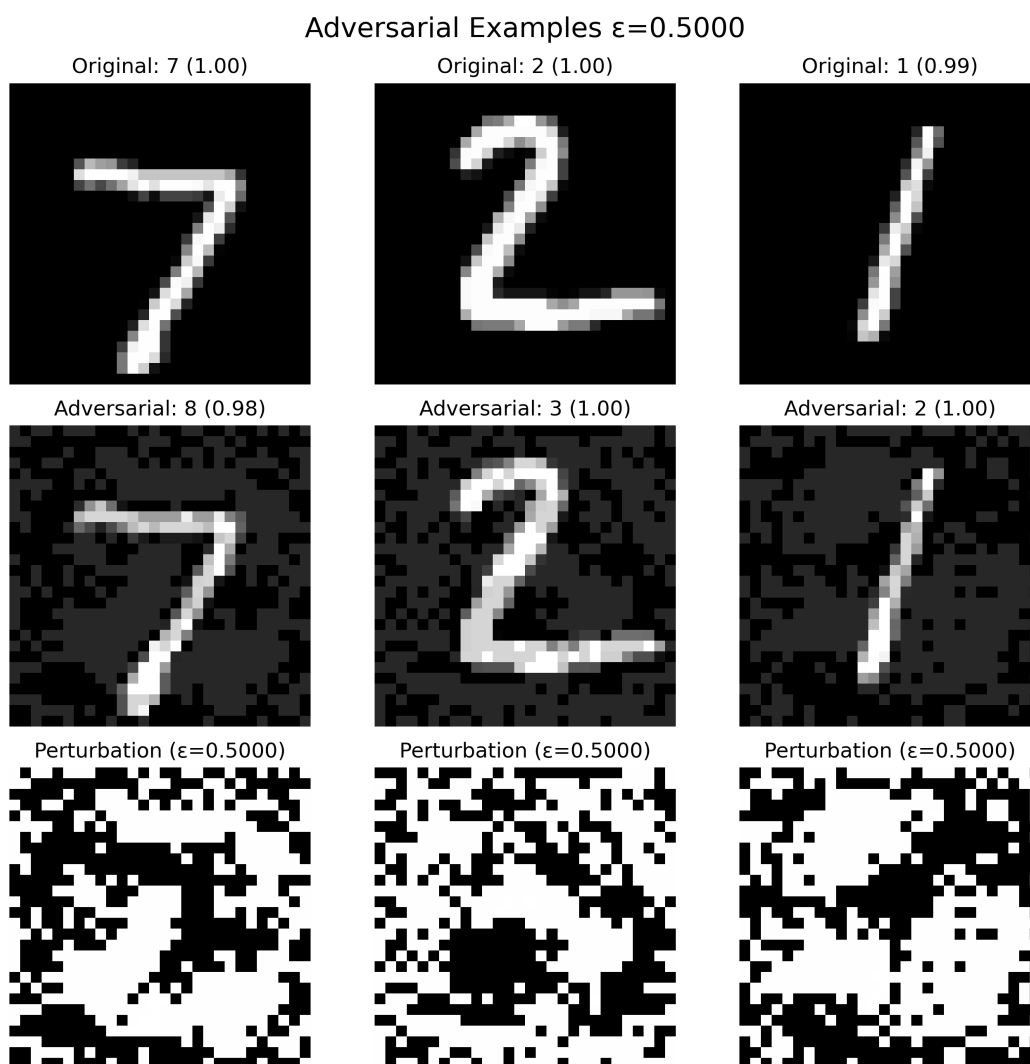
S obzirom na to da CIFAR-100 ima hijerarhijsku strukturu i slike klase unutar iste nadklase su međusobno sličnije, razumno je pretpostaviti da će uspješnost ciljanog napada ovisiti o tome koja je ciljna klasa. Očekujemo da će biti lakše postići uspješan napad ako je ciljna klasa slična originalnoj. Na primjer, puno je lakše "zamijeniti" jabuku i krušku nego jabuku i avion.

Zato smo odlučili napraviti napad za dva slučaja:

- ciljna klasa pripada istoj nadklasi kao originalna klasa
- ciljna klasa pripada različitoj nadklasi

Ciljna klasa pripada istoj nadklasi kao i originalna

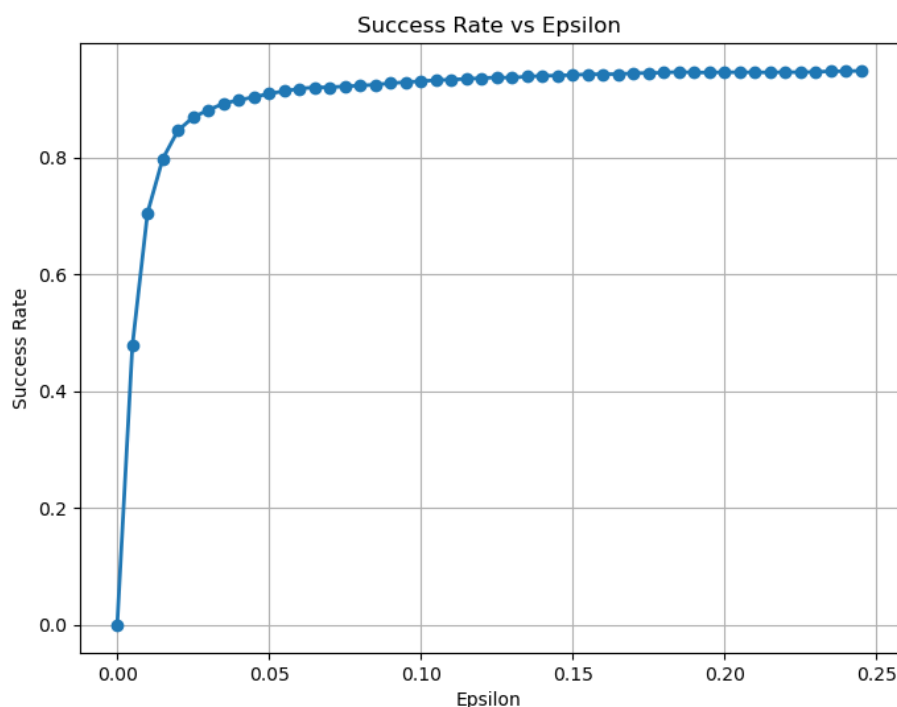
U ovom slučaju, tražimo perturbacije koje će model navesti na predikciju drugog razreda istog nadrazreda. Na primjer, CIFAR-100 ima nadklasu *veliki mesojedi (large carnivores)* u koju spadaju medvjed, leopard, lav, tigar i vuk. Dakle svaka ciljna klasa za klasu iz velikih mesojeda je isto neka (različita) klasa iz te iste nadklase. Konkretno, za primjer



Slika 4.9. Suparnički primjeri: MNIST, $\epsilon = 0.5$, Ciljani napad.

lava, želimo da model nakon dodavanja suparničke perturbacije da klasifikaciju medvjeda. Još neki primjeri: za klasu deva ciljna klasa je govedo, za klasu tuljan ciljna klasa je dabar, za klasu jabuka ciljna klasa je kruška itd. Neki od navedenih primjera prikazani su na slici 4.13., dok slika 4.12. prikazuje graf uspješnosti s obzirom na ϵ .

Zanimljivo je primjetiti kako u ovom slučaju uspješnost napada nakon početnog rasta počinje padati s povećanjem ϵ -a, za razliku od prethodno opaženih rezultata gdje je uspješnost uvijek rasla. Ovaj pad može se objasniti specifičnošću ciljanog napada. Model mora pogrešno klasificirati ulaz točno u unaprijed odabranu klasu, a prevelika perturbacija ($\epsilon > 0.03$) često "prebaci" primjer u neku treću, potpuno nepovezanu klasu. To se ne događa kod neciljanih napada jer je tamo svaka pogreška jednako vrijedna, niti kod MNIST-a gdje su klase jasnije odvojene. Stoga postoji optimalan ϵ (oko 0.03) gdje

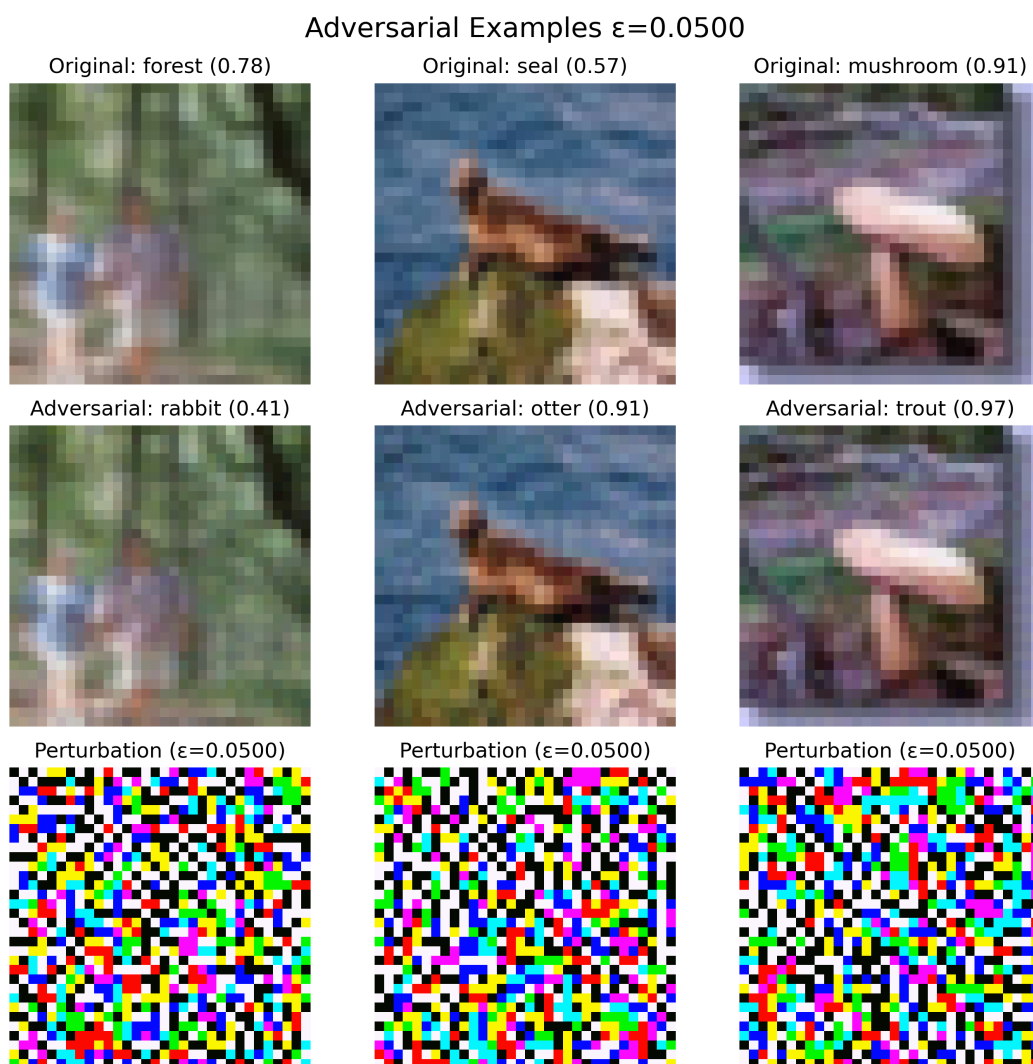


Slika 4.10. Uspješnost neciljanog napada na ispitnom podskupu skupa CIFAR-100.

je napad najprecizniji (uspješnost oko 30%) – dovoljno jak da prevari model onako kako mi želimo, ali ne toliko da izgubi kontrolu nad smjerom pogreške. Slika 4.13. prikazuje nekoliko primjera za tu vrijednost.

Ciljna klasa ne pripada istoj nadklasi kao i originalna

U ovom slučaju ciljna i originalna klasa su međusobno više različite. Slika 4.14. pokazuje kako će se to odraziti na graf uspješnosti. Iz slike 4.14. je vidljivo da opet postoji optimalna vrijednost ϵ za koju je napad najuspješniji i nakon koje uspješnost počinje padati. U ovom slučaju ta je vrijednost malo veća (oko 0.055) i uspješnost je isto manja (oko 15%). To je očekivano zato što su originalna i ciljna klasa međusobno dalje pa model mora napraviti veću pogrešku i za to je potrebno više promijeniti sliku. Veća promjena znači da je lakše završiti u području neke druge klase.



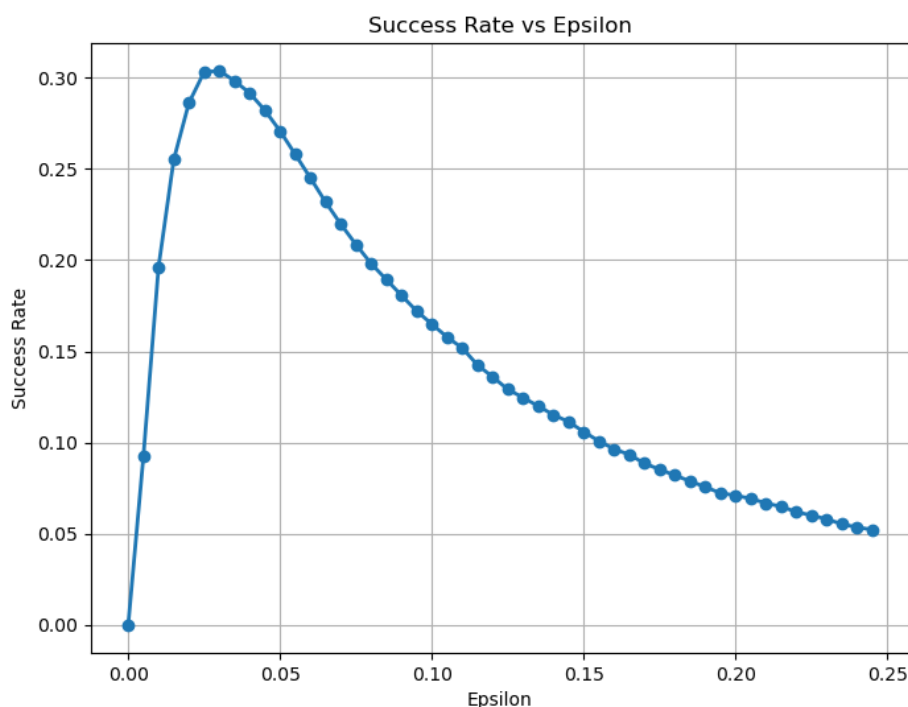
Slika 4.11. Suparnički primjeri: CIFAR-100, neciljani napad, $\epsilon = 0.05$

4.7. Usporedba robusnosti

4.7.1. Neciljani napad

Iz dobivenih rezultata može se zaključiti da za neciljani napad potpuno povezani model treniran na skupu MNIST pokazuje veću robusnost na suparničke napade u usporedbi s modelom ResNet-20 treniranim na skupu CIFAR-100. U prosjeku, ulazna slika potpuno povezanog modela (uzorak iz MNIST skupa) mora biti više promijenjena da bi izazvala krivu klasifikaciju nego ulazna slika za rezidualni konvolucijski model (uzorak iz CIFAR-100 skupa).

Važno je istaknuti da su ovi rezultati dobiveni u različitim eksperimentalnim uvjetima.

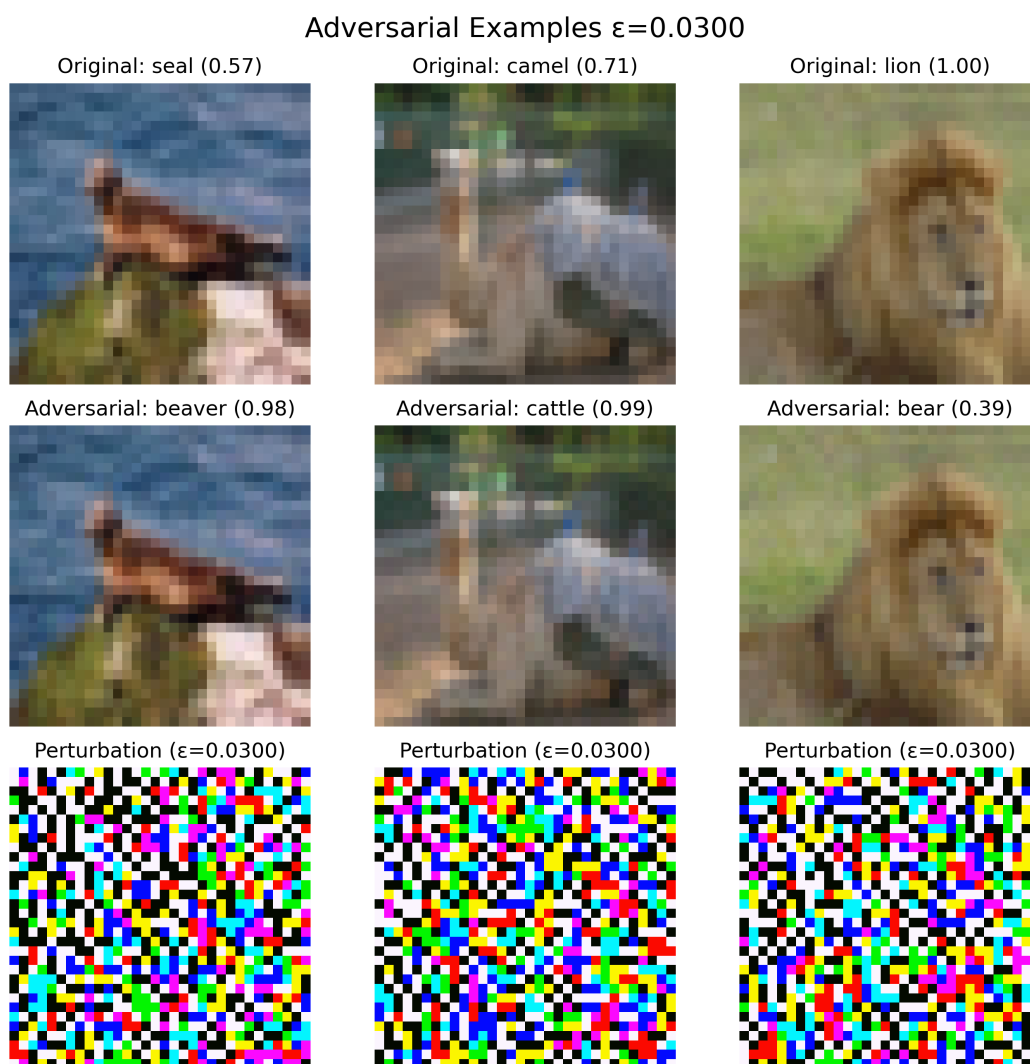


Slika 4.12. Uspješnost ciljanog napada na ispitnom podskupu skupa CIFAR-100; ciljna klasa pripada istoj nadklasi kao originalna.

Skup CIFAR-100 sastoji se od kompleksnijih, višedimenzionalnih ulaza s većim brojem klasa (100) u usporedbi s MNIST-om (10 klasa), što može utjecati na ponašanje modela pod napadom.

Intuitivno, veći broj klasa povećava vjerojatnost pogrešne klasifikacije jer postoji više mogućnosti za pogrešku. Rezultati iz literature podržavaju ovu pretpostavku, pokazujući da klasifikacijski zadaci s većim brojem klasa često rezultiraju smanjenom robusnošću modela na suparničke napade [16].

ResNet-20 na CIFAR-100 testnom skupu pokazuje nižu točnost nego potpuno povezani model na MNIST skupu. Budući da se suparnički napadi u ovom radu provode isključivo na ispravno klasificiranim podacima, to rezultira manjim brojem važećih primjera za analizu robusnosti. Iako različita veličina uzorka ne objašnjava samu razliku u robusnosti, važno je napomenuti da nije bio korišten identičan broj primjera u oba eksperimenta.

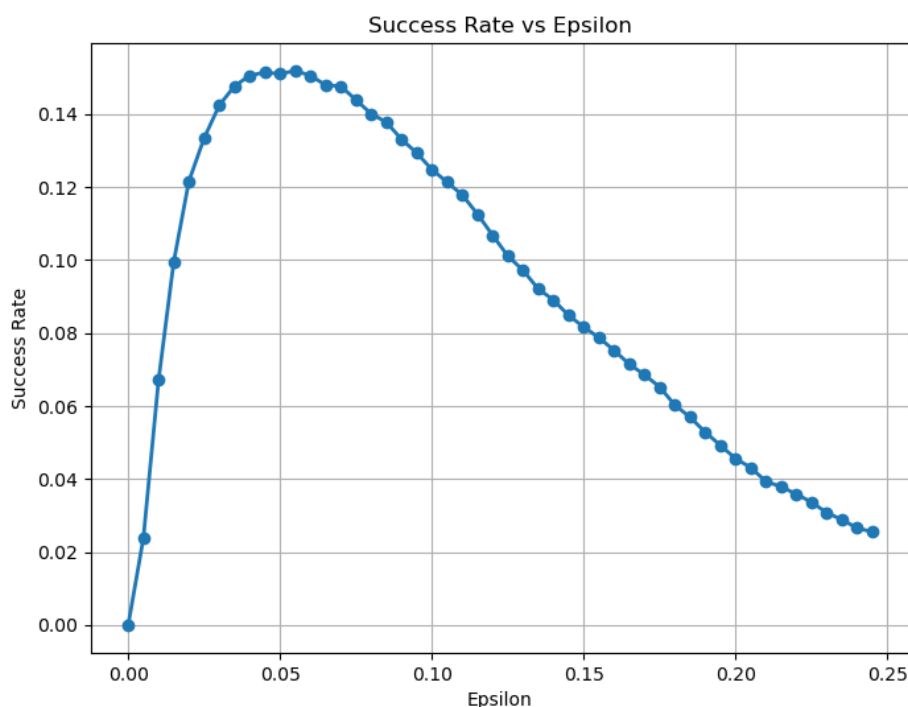


Slika 4.13. Suparnički primjeri: CIFAR-100, ciljani napad, ista nadklasa $\epsilon = 0.03$

4.7.2. Ciljani napad

Za ciljani napad je situacija obrnuta. Uspješni ciljani napad je puno lakše postići na modelu za MNIST klasifikaciju nego za CIFAR. Razlog za to smo već naveli u prethodnom poglavlju: graf uspješnosti za CIFAR model (4.12. i 4.12.) nakon dovoljno velikog ϵ -a kreće padati i daljnje povećavanje vrijednosti ϵ neće povećavati uspješnost. To nije slučaj kod MNIST modela kod kojeg daljnim povećavanjem ϵ -a povećavamo i uspješnost. Razlog za to je to što velike promjene u smjeru koji maksimizira vjerojatnost ciljne klase na MNIST slikama teže uzrokuju klasifikaciju u nepovezanu klasu. To je zato što MNIST ima manje klasa i one su jasnije odvojene.

Treba napomenuti i to da za ϵ vrijednosti za koje uspješnost napada na model za skup



Slika 4.14. Uspješnost ciljanog napada na ispitnom podskupu skupa CIFAR-100; ciljna klasa ne pripada istoj nadklasi kao originalna.

CIFAR još uvijek raste, napad ima puno veću uspješnost nego kod modela učenog na skupu MNIST. Međutim ta uspješnost nakon neke vrijednosti kod MNIST-a nastavlja rasti dok za CIFAR pada.

4.8. Nedovršeni pokušaji i napuštene ideje

Tijekom rada razvijani su i dodatni pristupi koji na kraju nisu uključeni u glavni dio rada zbog nedovršene implementacije, nepouzdanosti ili nezadovoljstva postignutim rezultatima. Ipak, s obzirom na vrijeme i trud uloženi u njihovu razradu te dopunjavanje konteksta istraživanja, vrijedi ih ukratko spomenuti.

Nakon uspješne implementacije FGSM metode razvijena je (djelomično) osnovna implementacija One Pixel Attack-a koji smo pokušali primjeniti na potupno povezani model treniran na skupu MNIST. Motivacija za to je bila da se uz FGSM, koji predstavlja black-box pristup i čini temelj rada, prikaže i primjer white-box napada. Implementiran je algoritam diferencijalne evolucije, koji je glavni dio napada. Međutim bilo bi po-

trebno dodatno raditi na njegovoj prilagodbi i ispravljanju potencijalnih grešaka kako bi se omogućila provedba relevantne analize i izvođenje zaključaka, kao što je to učinjeno za FGSM. Odlučeno je posvetiti se poboljšavanju FGSM metode i njena primjena na više modela umjesto dodavanja novih metoda koje možda zbog nedostatka vremena nebi bile na željenoj razini.

Još jedan pokušaj bio je treniranje klasifikacijskog modela na skupu MNIST pomoću generacijskog genetskog algoritma. Motivacija za ovaj pristup bila je isprobati treniranje koje se ne oslanja na gradijente kao kontrast treniranju sa sgd-om. Iako je očekivano da će takav pristup dati lošije rezultate, činilo se zanimljivim usporediti robusnost modela treniranih nekonvencionalnim metodama poput ove, jer bi se mogle otkriti neočekivane otpornosti na određene tipove napada. Tijekom više od 30 sati treniranja postignuta je točnost od približno 60% na MNIST testnom skupu, nakon čega daljnje poboljšanje nije bilo uočeno. Taj rezultat je relativno loš za MNIST skup podataka gdje smo s sgd-om postigli mnogo veću točnost. Razmatrala se ideja mijenjanja hiperparametara i operatora u algoritmu kako bi se potencijalno postiglo poboljšanje. Međutim, budući da je tema rada usmjerena na suparničke napade, ova je ideja napuštena.

5. zaključak

U ovom radu istraženi su suparnički napadi na klasifikacijske modele slika. Posebna je pažnja posvećena napadu Fast Gradient Sign Method (FGSM). Implementiran je vlastiti sustav koji uključuje treniranje potpuno povezanog modela algoritmom stohastičkog gradijentnog spusta (SGD) na MNIST skupu te izvođenje napada na taj model i na unaprijed istrenirani ResNet-20 model za CIFAR-100.

Rezultati pokazuju da potpuno povezani model za skup MNIST pokazuje veću robusnost na neciljane FGSM napade, dok ResNet-20 za skup CIFAR-100 pokazuje veću robusnost na ciljane napade.

Implementirani sustav omogućuje pouzdanu i konzistentnu evaluaciju suparničkih napada te praćenje utjecaja perturbacija na točnost modela. Ovaj rad predstavlja osnovu za daljnja istraživanja koja mogu uključivati izvođenje napada na druge modele i skupove podataka, kao i integraciju obrambenih tehnika s ciljem ispitivanja njihove učinkovitosti u različitim uvjetima.

Literatura

- [1] D. E. Rumelhart, G. E. Hinton, i R. J. Williams, “Learning representations by back-propagating errors”, *Nature*, sv. 323, br. 6088, str. 533–536, 1986.
- [2] Y. LeCun, L. Bottou, Y. Bengio, i P. Haffner, “Gradient-based learning applied to document recognition”, *Proceedings of the IEEE*, sv. 86, br. 11, str. 2278–2324, 1998.
- [3] A. Krizhevsky, I. Sutskever, i G. E. Hinton, “Imagenet classification with deep convolutional neural networks”, u *Advances in Neural Information Processing Systems 25: 26th Annual Conference on Neural Information Processing Systems 2012. Proceedings of a meeting held December 3-6, 2012, Lake Tahoe, Nevada, United States*, 2012., str. 1106–1114.
- [4] S. Šegvić, “Predavanja iz dubokog učenja”, 2016.
- [5] K. He, X. Zhang, S. Ren, i J. Sun, “Deep residual learning for image recognition”, u *2016 IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2016, Las Vegas, NV, USA, June 27-30, 2016*. IEEE Computer Society, 2016., str. 770–778.
- [6] —, “Identity mappings in deep residual networks”, u *Computer Vision - ECCV 2016 - 14th European Conference, Amsterdam, The Netherlands, October 11-14, 2016, Proceedings, Part IV*, ser. Lecture Notes in Computer Science, B. Leibe, J. Matas, N. Sebe, i M. Welling, Ur., sv. 9908. Springer, 2016., str. 630–645.
- [7] K.-L. Chung i Y.-L. Chang, “An effective backward filter pruning algorithm using k1,n bipartite graph-based clustering and the decreasing pruning rate approach”, *TechRxiv*, br. 15642330, 08 2021. <https://doi.org/10.36227/techrxiv.15642330>

- [8] I. J. Goodfellow, J. Shlens, i C. Szegedy, “Explaining and harnessing adversarial examples”, u *3rd International Conference on Learning Representations, ICLR 2015, San Diego, CA, USA, May 7-9, 2015, Conference Track Proceedings*, Y. Bengio i Y. LeCun, Ur., 2015.
- [9] A. Madry, A. Makelov, L. Schmidt, D. Tsipras, i A. Vladu, “Towards deep learning models resistant to adversarial attacks”, u *6th International Conference on Learning Representations, ICLR 2018, Vancouver, BC, Canada, April 30 - May 3, 2018, Conference Track Proceedings*, 2018.
- [10] N. Carlini i D. A. Wagner, “Towards evaluating the robustness of neural networks”, u *2017 IEEE Symposium on Security and Privacy, SP 2017, San Jose, CA, USA, May 22-26, 2017*, 2017., str. 39–57.
- [11] C. Szegedy, W. Zaremba, I. Sutskever, J. Bruna, D. Erhan, I. J. Goodfellow, i R. Fergus, “Intriguing properties of neural networks”, u *2nd International Conference on Learning Representations, ICLR 2014, Banff, AB, Canada, April 14-16, 2014, Conference Track Proceedings*, Y. Bengio i Y. LeCun, Ur., 2014.
- [12] J. Su, D. V. Vargas, i K. Sakurai, “One pixel attack for fooling deep neural networks”, *CoRR*, 2017. [Mrežno]. Adresa: <http://arxiv.org/abs/1710.08864>
- [13] A. Athalye, N. Carlini, i D. A. Wagner, “Obfuscated gradients give a false sense of security: Circumventing defenses to adversarial examples”, u *Proceedings of the 35th International Conference on Machine Learning, ICML 2018, Stockholmsmässan, Stockholm, Sweden, July 10-15, 2018*, ser. *Proceedings of Machine Learning Research*, J. G. Dy i A. Krause, Ur., sv. 80. PMLR, 2018., str. 274–283.
- [14] H. Salman, S. Jain, E. Wong, i A. Madry, “Certified patch robustness via smoothed vision transformers”, u *IEEE/CVF Conference on Computer Vision and Pattern Recognition, CVPR 2022, New Orleans, LA, USA, June 18-24, 2022*. IEEE, 2022., str. 15 116–15 126.
- [15] Wikipedia contributors, “Mnist dataset example”, https://en.wikipedia.org/wiki/File:MNIST_dataset_example.png, accessed: June 1, 2025.

- [16] S. Qian, D. Kalimeris, G. Kaplun, i Y. Singer, “Robustness from simple classifiers”, *arXiv preprint arXiv:2002.09422*, 2020.

Sažetak

Suparnički napadi na modele za klasifikaciju slika

Tonka Šegvić

Ovaj rad istražuje suparničke napade na modele za klasifikaciju slika, s posebnim naglaskom na metodu Fast Gradient Sign Method (FGSM). Cilj rada bio je analizirati ranjivost modela na male perturbacije koje uzrokuju pogrešne klasifikacije. U praktičnom dijelu rada izradili smo programsku implementaciju koja omogućuje izvođenje FGSM napada nad različitim modelima i skupovima podataka. Implementirani napad je potom primjenjen na potpuno povezani model za klasifikaciju skupa podataka MNIST te model ResNet-20 za klasifikaciju skupa CIFAR-100. Osim implementacije FGSM-a, implementiran je i algoritam stohastičkog gradijentnog spusta (SGD), korišten za treniranje potpuno povezanog modela na skupu MNIST na koji se kasnije primjenjivao napad. Rezultati pokazuju da potpuno povezani model pokazuje veću robusnost na neciljane napade, dok je ResNet-20 otporniji na ciljane napade. Rad naglašava važnost razumijevanja suparničkih napada za razvoj sigurnijih i robusnijih modela umjetne inteligencije.

Ključne riječi: suparnički napadi, FGSM, duboko učenje, klasifikacija slika, robusnost, MNIST, CIFAR-100, ResNet-20, ciljani napadi, neciljani napadi

Abstract

Adversarial attacks on image classification models

Tonka Šegvić

This paper explores adversarial attacks on image classification models, focusing on the Fast Gradient Sign Method (FGSM). The main goal was to examine how small perturbations can fool models into making incorrect predictions. In the practical part of the work, we developed and tested a software implementation that enable FGSM attacks on various models and datasets. We applied the attack to a fully connected model trained on the MNIST dataset and to a pretrained ResNet-20 model for CIFAR-100 dataset. We also implemented a Stochastic Gradient Descent (SGD) algorithm which was used to train the fully connected model. Our results show that the fully connected model resists untargeted attacks more effectively, while ResNet-20 performs better against targeted attacks. This work highlights the need to understand adversarial attacks in order to build more secure and reliable AI systems.

Keywords: adversarial attacks, FGSM, deep learning, image classification, robustness, MNIST, CIFAR-100, ResNet-20, targeted attacks, untargeted attacks